

4 分布式计算

冯奕
2026



南京大學
NANJING UNIVERSITY

目录

- 分布式计算
 - 分布式计算概述
 - 分布式计算理论基础
- 分布式系统
 - 特性
 - 类型
- 分布式计算、存储和资源管理系统
 - 案例1: Hadoop
 - 案例2: Spark
 - 案例3: 阿里开放数据处理平台



分布式计算—概述（1）

- 集中式计算

- 完全依赖一台大型的中心计算机的处理能力，即主机，与其相连的终端设备具有各不相同**非常低的计算能力**。实际上大多数终端完全不具有处理能力，**仅作为输入输出设备使用**。

- 分布式计算

- 多个通过**网络互联的计算机**都具有一定的计算能力，他们相互之间传递数据，实现信息共享，协作共同完成一个处理任务。

- 中国科学院：分布式计算就是**在两个或多个软件互相共享信息**，这些软件既可以在同一台计算机上，也可以在通过网络连接起来的多台计算机上运行

- 优势

- 稀有资源实现共享；
- 在多台计算机上平衡计算负载；
- 将程序放在最适合它的计算机上运行。



分布式计算—概述（2）

- 分布式系统（distributed system）

- 是建立在网络之上的软件系统
- 正是因为软件的特性，所以分布式系统具有高度的内聚性和透明性。



每一个节点
高度自治

管理者最小化

高内聚是指将系统中相关的功能或模块组织在一起，使它们形成一个紧密耦合的单元，该单元内部的各个部分相互依赖，协同完成一定的功能。高内聚的模块在内部紧密配合，对外部模块的影响和干扰相对较小，能够提高系统的可靠性和可维护性，减少系统出现故障的概率。

- 网络和分布式系统之间的区别更多的在于高层软件（特别是操作系统），而不是硬件。
- 在一个分布式系统中，一组独立的计算机展现给用户的是一个统一的整体，就好像是一个系统似的。

用户体会不到与单机系统的差别：透明性

- 系统拥有多种通用的物理和逻辑资源，可以动态的分配任务，分散的物理和逻辑资源通过计算机网络实现信息交换。



分布式计算—概述（3）

- 分布式系统（distributed system）
 - 系统中存在一个以全局的方式管理计算机资源的分布式操作系统。
 - 将海量计算能力才能处理的问题拆分成许多小块
 - 将小块分配给同一套系统中不同的计算机节点处理
 - 最后将分开计算的结果合并得到最终结果的系统。
 - 通过网络消息实现数据通信与协调
- **1) 可用性（容错性）**：分布式计算系统中的一个重要的优点是可靠性。一台服务器的系统崩溃并不影响到其余的服务器，仍可以正常对外提供服务。
- **2) 可扩展性**：可以通过线性的增加机器资源，来应对不断增长的外部需求。
- **3) 资源共享**：共享数据是必不可少的应用，如银行，预订系统。
- **4) 灵活性**：由于该系统是非常灵活的，它很容易安装，实施和调试新的服务。
- **5) 更快的速度**：多地部署，将用户请求按地理路由到最近机房处理。拥有多台计算机的计算能力，使得它比其他系统有更快的处理速度
- **6) 开放系统**：由于它是开放的系统，本地或者远程都可以访问到该服务。
- **7) 更高的性能**：相较于集中式计算机网络集群可以提供更高的性能（及更好的性价比）。



分布式计算—概述（4）

- 分布式计算的一般步骤
 - 设计分布式计算模型，实现分布式系统；
 - 通过分布式系统实现分布式任务分配；
 - 依照分布式系统编写并执行分布式程序

如何将复杂算法优化分解为适用于多个节点分别计算的小任务：
特别是希望节点之间互不相干



- 【难点：计算任务划分 + 多节点通信】



多数还是要相互通信的：
消息队列；分布式存储



分布式计算—理论基础（1）

- 分布式系统最大的问题是复杂性

- 1、数据的一致性

- 考虑到大量的机器故障：宕机、重启、关机，数据可能丢失、陈旧、出错，如何让系统容纳这些问题，对外保证数据的正确性，需要相当复杂的设计。

- 2、网络和通信故障

- 网络的不可靠，消息可能丢失、早到、迟到、Hang住，这给机器间的协调带来了极大的复杂度。像TCP等网络基础协议，能解决部分问题，但更多的需要系统层面自己处理。更不用说，开放式网络上可能存在的消息伪造。

- 3、管理复杂度

- 机器数量到达一定数量级时，如何对他们进行有效监控、收集日志、负载均衡，都是很大挑战。

- 4、延迟

- 网络通信延迟要比机器内通信高出几个数量级，而组件越多、网络跳数越多，延迟便会更高，这些最终都会作用于系统对外服务质量上。

- CAP理论：一个**分布式系统**最多能够同时满足一致性(consistency)、可用性(Availability)、分区容错性(Partition tolerance)中的两项

- 一致性——一次操作之后，所有节点同一时间的数据完全一致。 强一致性 弱一致性 最终一致性
 - 可用性——服务一直可用且在正常的响应时间内。哪怕只有极短的时间不可用也不行。
 - 分区容错性——分布式系统遇到某节点或网络分区故障时，仍然能够正常运行并对外提供服务。

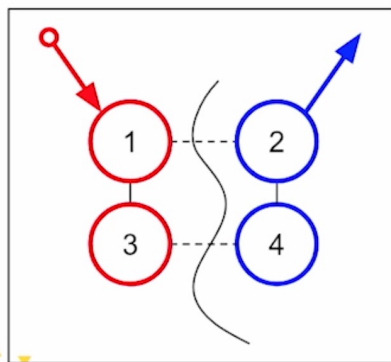


分布式计算—理论基础 (2)

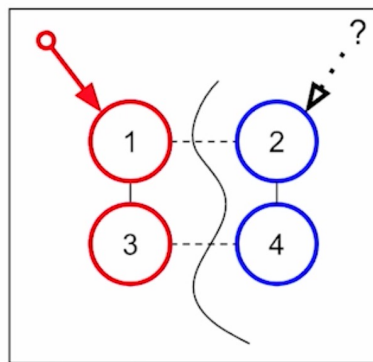
- CAP理论：一个**分布式系统**最多能够同时满足一致性(consistency)、可用性(Availability)、分区容错性(Partition tolerance)中的两项

- 阐述与证明

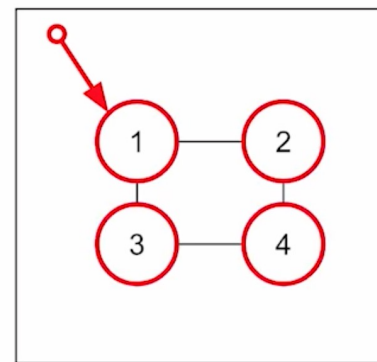
Partition Tolerant + Available = **Not Consistent**



Partition Tolerant + Consistent = **Not Available**



Consistent + Available = **Not Partition Tolerant**



- 如何取舍?

- CA: 但是分区始终存在, 保证子系统CA
- CP: 导致某些节点无法使用
- AP: 导致全局数据不一致, 但是高可用

对于大多数大型互联网服务而言, 节点故障、网络故障是常态, 均采取保证**AP**的策略, 对于一致性退而求其次, 只保证最终一致性

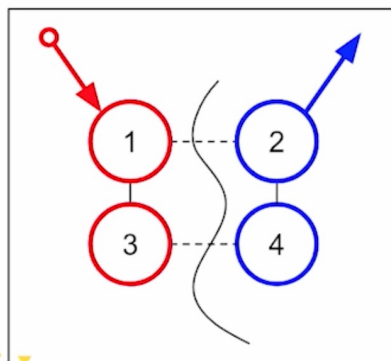


分布式计算—理论基础 (2)

- CAP理论：一个**分布式系统**最多能够同时满足一致性(consistency)、可用性(Availability)、分区容错性(Partition tolerance)中的两项

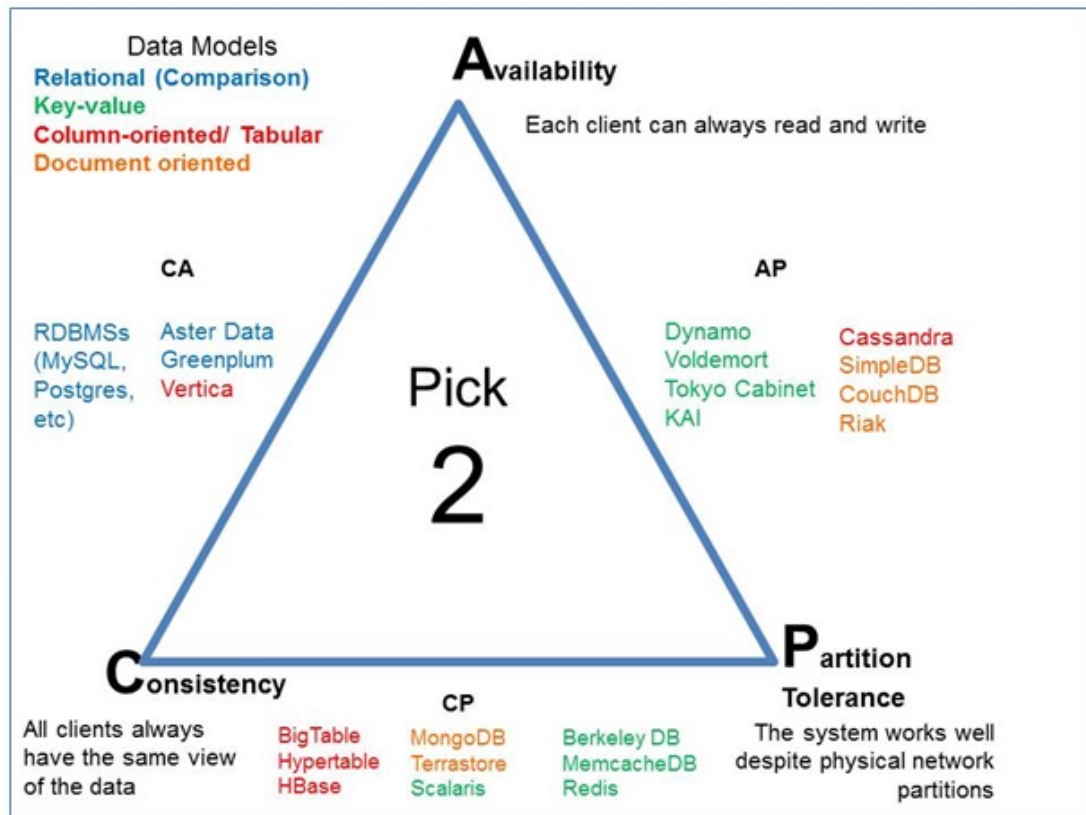
- 阐述与证明

Partition Tolerant + Available = **Not Consistent**



- 如何取舍?

- CA: 但是分区始终存在, 保证子系统CA
- CP: 导致某些节点无法使用
- AP: 导致全局数据不一致, 但是高可用



对于大多数大型互联网服务而言, 节点故障、网络故障是常态, 均采取保证AP的策略, 对于一致性退而求其次, 只保证最终一致性



分布式计算—理论基础（3）

- BASE理论——追求最终一致性
 - Basically Available: 基本可用-系统出现故障时, 允许损失部分可用性, 保证核心可用
 - Soft State: 软状态: 允许系统存在中间状态, 但中间状态不会影响系统的整体可用性
 - Eventual Consistency: 最终一致性-所有数据副本经过一定时间后, 能最终达到一致的状态
- 一致性算法（共识算法）
 - 一个典型的场景是, 在一个分布式数据库系统中, 如果各节点的初始状态一致, 每个节点执行相同的操作序列, 那么他们最后能得到一个一致的状态。为保证每个节点执行相同的命令序列, 需要在每一条指令上执行一个“一致性算法”以保证每个节点看到的指令一致。
 - 一台机器中多个进程/线程达成数据一致;
 - 分布式文件系统或者分布式数据库中多客户端并发读写数据;
 - 分布式存储中多个副本响应读写请求的一致性。
 - 基于消息传递的一致性算法Paxos: <https://www.cnblogs.com/linbingdong/p/6253479.html>
 - 基于消息传递的一致性算法Raft: <https://raft.github.io/>



Paxos算法——具体过程

- Basic Paxos

- 角色

- 提议者Proposer: 向所有其他节点提出提案, 提案 $\text{Proposal} = (\text{Proposal ID}, \text{Value})$
 - 分为两个步骤
 - 1. 准备: 向所有其他决策者发送提案ID, 等待回复
 - 2. 收到多数决策者的回应后, 再发送提案Value
 - 决策者Acceptor: 回应提议者的提议, 根据自身状态有不同的回应方式
 - 如果已经收到过其他提议者的提议, 则将提案ID和对应的Value返回
 - 如果没有收到过提议, 则返回空
 - 决策学习者Learner: 不参与决策, 只是从决策过程学习到最终的一致提案的Value

- 过程: 分为三个阶段

- 1. Proposer准备; Acceptors回应, 并许下承诺
 - 2. Proposer收到多数回应后, 发出Value; Acceptors进行Accept处理
 - 3. Proposer收到多数回应后, 表示提案成功, 立即将决议发送给所有Learner

承诺什么?

Value如何决定?



Paxos算法——细节

- 承诺什么？

- 不再接收比当前提案ID小的或相同的提案的准备Prepare请求
- 不再接收比当前提案ID小的提案的决策Accept请求

ID代表时间：带时间戳
只会响应更新的提案

获得本轮提案的Value

- 提议者的Value如何确定

- 收到决策者的承诺后，从应答中选择提案ID最大的那个提案的Value作为本次要发起的提案Value
- 如果所有应答都为空，则可以自己随意决定提案的Value

- 如何确定被选中的值

- 当Value发给决策者后，决策者判断是否是自己已经承诺过的那个提议，如果是则接受，并返回消息给提议者
- 提议者收到多数人的回复后，才能确定这个值为一致选中的决策

只有提议者才知道最终被选中的值是什么，而决策者自己无法知道！

为什么？



Paxos算法——分析

假设有 3 个 Acceptor: A1, A2, A3。

1. Proposer P1 提议值 v1, 编号 n=1。
2. P1 向 A1、A2、A3 发送 Prepare(1)。
3. 所有 Acceptor 承诺 (Promise), 返回空 (未接受过旧值)。
4. P1 发送 Accept Request(1, v1)。
5. A1、A2、A3 接受, v1 被选择。
6. Learner 得知 v1 成为最终值。

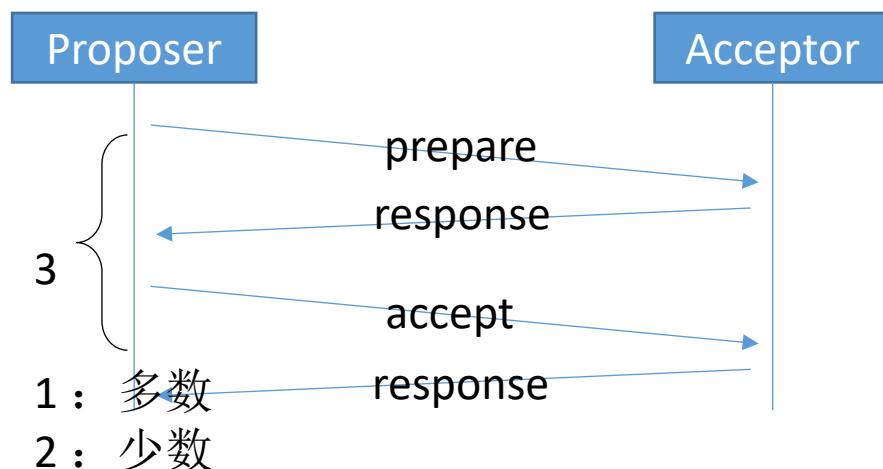
如果此时另一个 Proposer P2 编号 n=2 提议 v2:

1. P2 发送 Prepare(2)。
2. Acceptor 收到后发现自己已经承诺过 n=1, 并可能已接受 v1, 因此会返回 v1。
3. P2 必须提议 v1, 保证安全性。
4. 最终 v1 再次被多数接受, v2 被忽略。



Paxos算法——分析

- 当有一个提议者发起一个提议后，之后再提议者发起提议可能的情况有：



- 1. 之前这个提议者的值已经被多数接受了，即已经被选中
 - 后来的提议者如果是收到多数回复，一定能够学到这个值，并作为自己的提议
 - 后来提议者没有收到多数回复，则直接失败
- 2. 之前这个值没有被选中，但是已经有人Accept了这个值
 - 后来提议者恰巧学到这个值，则作为自己的提议
 - 后来提议者没有看到这个值被接收，则使用自己的值，之前的值失效了
- 3. 之前这个值没有被选中，也没有任何人Accept了这个值
 - 响应新提议者的节点会承诺不再响应第一个提议者的任何信息



分布式计算—核心算法（1）

• Basic Paxos

- 提议者Proposer：向所有其他节点提出提案
 - 1. 准备：向所有其他决策者发送提案ID，等待回复
 - 2. 收到多数决策者的回应后，再发送提案Value
- 决策者Acceptor：回应提议者的提议
 - 如果已经收到过提议，则将ID和对应的Value返回
 - 如果没有收到过提议，则返回空
- 学习者Learner：不参与决策，只是从决策过程学习到最终的一致提案的Value

Proposer	Acceptor
$last_prepare_b = 0, maxb = 0$	$pb = 0, ab = 0, av = null$
$b = last_prepare_b + 1$ $maxb = 0, v = \text{☺}$ Prepare (b)	OnPrepare (b): if $b \geq pb$: $pb = b, \text{Prepare_Response}(ok, ab, av)$ else Prepare_Response (<i>reject</i> , 0, null)
OnPrepare_Response (ok, ab, av): if $ab > maxb$ and $av \neq null$: $maxb = ab, v = av$ if $count(ok) \geq count(qrm)$: Accept (b, v)	OnAccept (b, v): if $b == pb$: $ab = b, av = v, \text{Accept_Response}(ok)$ else Accept_Response (<i>reject</i>)
OnAccept_Response (ok): if $count(ok) \geq count(qrm)$: done ()	

• 过程：分为三个阶段

- 1. Proposer发准备请求；Acceptors回应ID和Value，并许下承诺
 不再接收比当前提案ID小的或相同的提案的准备Prepare请求和Accept请求
- 2. Proposer收到多数回应后，发出带Value的Accept请求；Acceptors进行Accept处理
 从应答中选择提案ID最大的那个提案的Value作为本次要发起的提案Value
- 3. Proposer收到多数回应后，表示提案成功，立即将决议发送给所有Learner

如果所有应答都为空，则可以自己随意决定提案的Value

能够有效解决选择主服务器的问题和保证大家都一致写入操作的问题



分布式计算—核心算法（2）

- Raft：对 Paxos 的工程化重构
 - 引入了**强领导者（Strong Leader）机制**，将原本分散在多个提议者之间的决策过程集中到单一领导节点，从而显著降低了状态空间复杂度与实现难度。
- 核心角色：
 - Leader（领导者）：系统中的唯一决策节点，负责接收客户端请求、生成日志条目并将其复制到其他节点，同时维护系统的整体进度与一致性。
 - Follower（跟随者）：被动节点，仅响应来自 Leader 或 Candidate 的请求（如日志复制或投票请求），自身不主动发起决策。
 - Candidate（候选者）：在选举阶段临时产生的角色，当节点在一定时间内未接收到 Leader 的心跳消息时，会转变为 Candidate 并发起新一轮领导者选举。



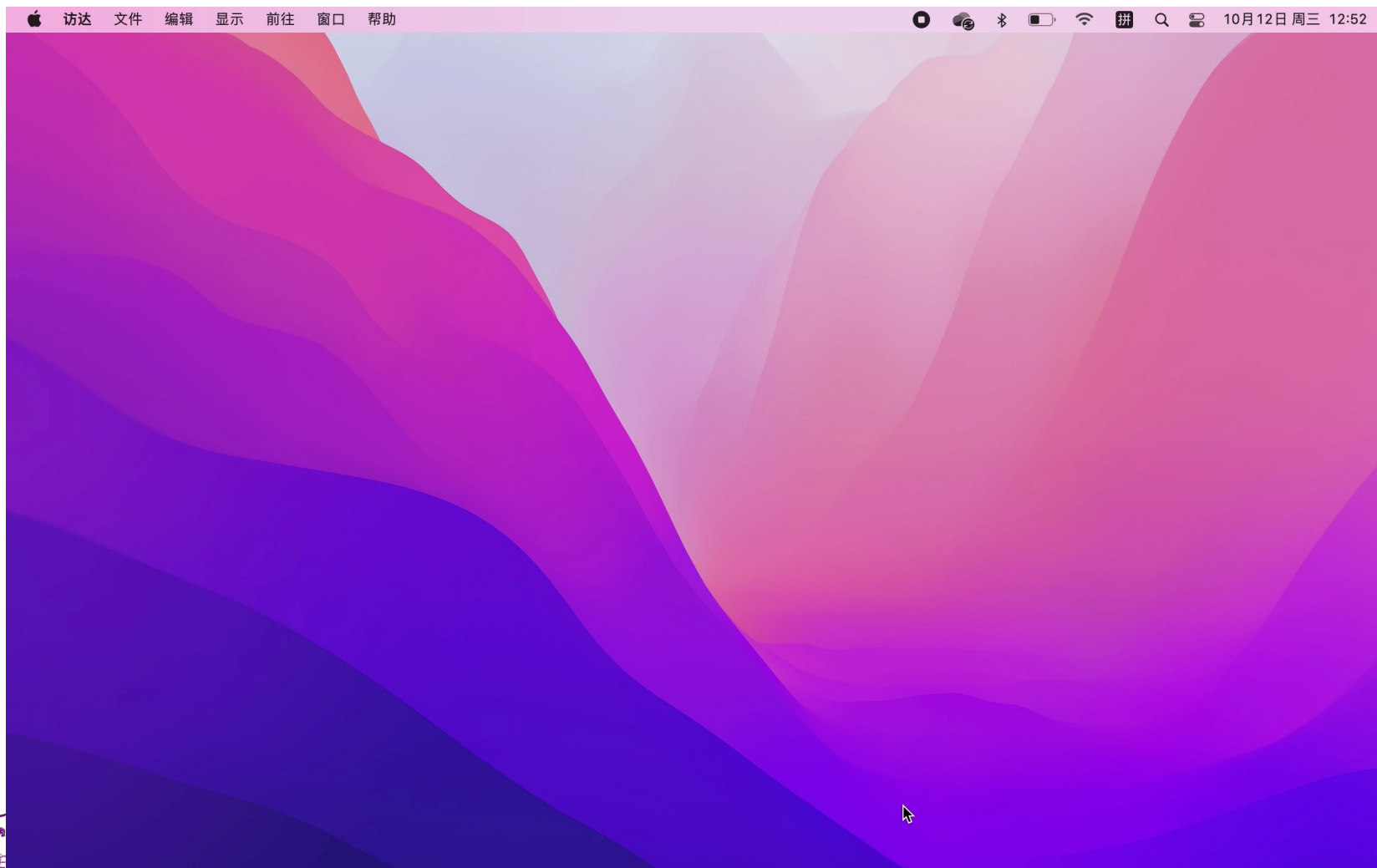
分布式计算—核心算法（3）

- 工作流程：
 - 领导者选举 (Leader Election)：选出系统中唯一的决策节点。
 - 所有节点初始为 Follower，等待心跳；
 - 超时未收到心跳的节点变为 Candidate，发起投票请求；
 - 获得超过半数票的 Candidate 成为 Leader，并发送心跳建立权威。
 - 日志复制 (Log Replication)：确保所有节点以相同顺序执行操作，实现状态机一致性。
 - 客户端请求由 Leader 接收并写入本地日志；
 - Leader 将日志复制到多数 Follower；
 - 当超过半数节点写入成功时，Leader 将日志标记为已提交，并通知 Followers 执行；
 - Follower 按顺序执行提交的日志，保持状态一致。
 - 异常处理与系统恢复：保证系统在 Leader 挂掉、日志不一致或网络分区时仍能正常运行。
 - Leader 失效 → Follower 超时 → 重新选举新 Leader；
 - 日志不一致 → Follower 覆盖为 Leader 的日志；
 - 网络分区 → 只允许多数派继续选举 Leader，避免冲突写入。



分布式计算—核心算法（4）

- Raft: <http://thesecretlivesofdata.com/raft/>

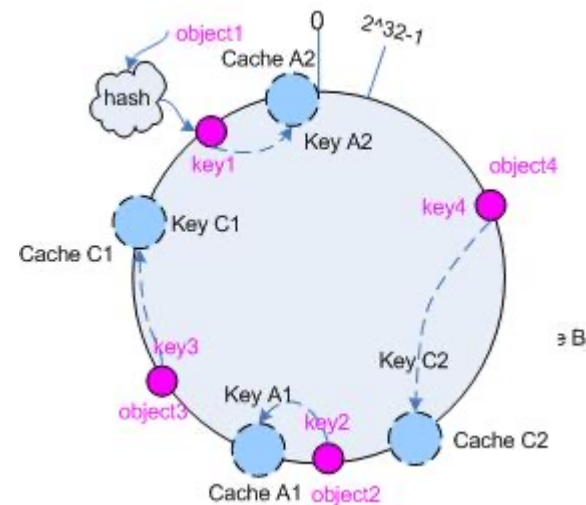
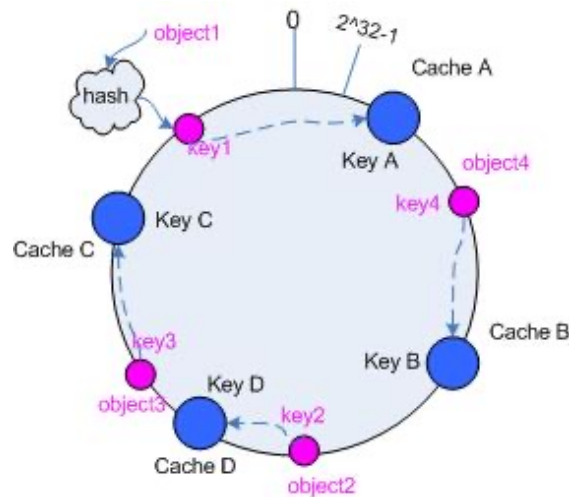
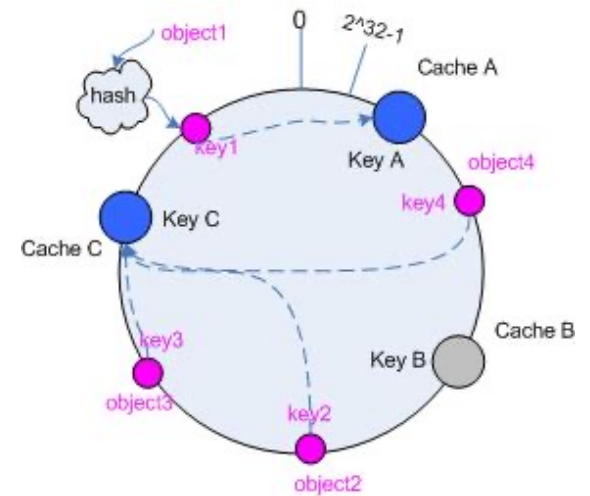
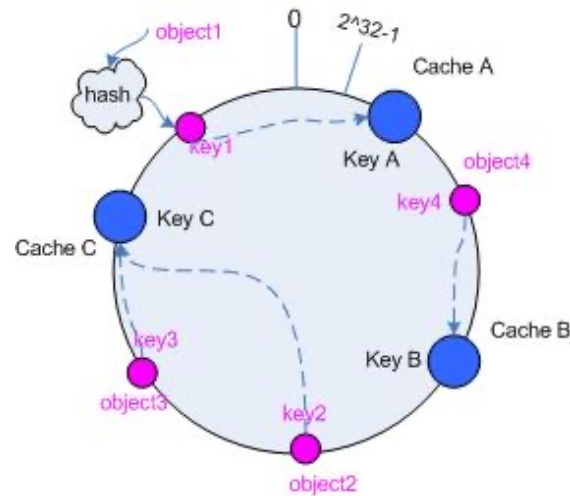
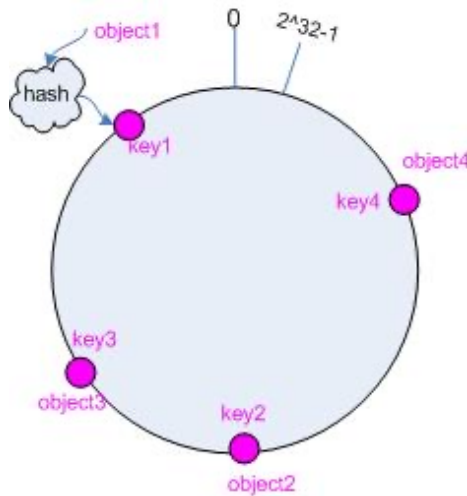


分布式计算—核心算法（5）

- 普通Hash算法的问题——Hash函数和Cache个数有关，Cache是固定的
 - $\text{CacheIndex} = \text{Hash}(\text{Key}) \bmod N$
 - 当有服务器失效或添加更多的服务器时，同一个Key的原有Hash结果和新Hash函数结果不一致，全部要重新计算
 - 希望后来添加的节点能够接收到更多的Key映射
- 一致性Hash算法
 - 将Cache和资源视为同等概念，都需要Hash，Hash和Cache数量无关
 - 单调性
 - 如果已经有一些内容通过哈希分派到了相应的缓冲中，又有新的缓冲加入到系统中，哈希的结果应能够保证原有已分配的内容可以被映射到新的缓冲中去，而不会被映射到旧的缓冲集合中的其他缓冲区。——【尽可能小的改变已存在 key 映射关系】
 - 平衡性
 - 平衡性是指哈希的结果能够尽可能分布到所有的缓冲中去，这样可以使得所有的缓冲空间都得到利用。——【负载均衡】； **使用虚拟节点**
 - 分散性、负载：同一个内容应该映射到同一个Cache，同一个Cache应该包含相同内容
 - 平滑性：服务器数量平滑改变和数据量平滑改变是一致的？



分布式计算—核心算法 (6)



分布式计算—核心算法（7）

- 优点
 - 只需要给定资源的Key就能够在网络中查找资源具体数据，而不需要指定资源IP等网络信息
 - 能够很好的共享下层设施，形成一个统一的基于内容的命名层，增加了寻找对象的灵活性
 - 网络时自组织和自治的，降低了执行、维护和管理代价
- 对节点要求
 - 自愿提供PC资源、网络带宽，能够接收任何类型的数据
- 所有的DHT路由算法包括三个方面
 - 如何Hash：对散列空间的描述
 - 各个节点如何分配管理散列空间，散列后的信息如何确定其存储节点位置
 - 路由发现算法：查询时如何高效地路由到存储目标信息的节点
- 具体实现：Chord, Pastry, Kademlia



目录

- 分布式计算
 - 分布式计算概述
 - 分布式计算理论基础
- 分布式系统
 - 特性
 - 类型
- 分布式计算、存储和资源管理系统
 - 案例1: Hadoop
 - 案例2: Spark
 - 案例3: 阿里开放数据处理平台



分布式系统—特性

- 容错性
 - 我们可能永远也造不出永远都不出故障的机器，更加难以开发出永不出错的软件，因为软件在一定程度上还依赖硬件的可靠性
 - 在大规模分布式系统中完全检测和避免所有可能发生的故障是不可能的，因此需要系统能够在某些节点发生故障的情况下，利用容错机制避免整套系统服务都不可用
- 高可扩展性
 - 能够在运行过程中自由地对系统内部节点或现有功能进行扩充，而不影响现有服务的运行
- 开放性
 - 开放性决定给了一个系统是否具备自我扩展和与其他系统集成能力；开放的接口+接口遵循协议=更好
- 并发处理能力
 - 分布式系统必须保证对象的操作在并发环境中能够安全的使用，保证数据一致性和系统高可用性
- 透明性
 - 不需要让用户知晓分布式系统的内部细节，暴露给用户访问资源和服务的方式即可，用户将系统看作是一个整体。



分布式系统—类型（1）

- 分布式存储系统

- 结构化存储：事务处理系统或关系型数据库，数据划分为表、字段和表关系，如分布式MySQL
- 非结构化存储：强调很高的可扩展性，存储数据非常自由，典型代表是分布式文件系统，如HDFS，GFS等
- 半结构化存储：为了解决非结构化数据随机访问性能差的问题，例如NoSQL，Key-Value Store，对象存储
- In-memory存储：基于内存的存储系统，利用内存实现极高读写性能，例如Memcached和Redis，做缓存
- NewSQL：既具备结构化存储的ACID事务支持，又拥有NoSQL半结构化存储的强大可扩展能力



分布式系统—类型 (2)

- 分布式计算系统

- 传统基于消息的系统：MPI (Message Passing Interface) 非常灵活，对应用程序无约束

- Dataflow系统：将计算抽象为高层算子，算子组合为有向无环图，由后端调度引擎并行化调度执行

- Hadoop: MapReduce;

- Spark: 更多类型的算子

框架对程序的结构有严格的约束：
算子、输入和输出等

- 流式计算、图计算、分布式机器学习——Spark都实现了这些类型的分布式计算



分布式系统—类型（3）

- 分布式资源管理系统

- **特征：**支持多种计算框架、高可扩展、高容错、高资源利用率、细粒度资源分配
- Yarn：Hadoop 2.0版本，解决了原来Hadoop扩展性较差的问题，可以在框架下自定义算子
- Apache Mesos：加州大学伯克利分校的一个研究项目，现在属于Apache基金会的一个项目
- Spark Standalone：Spark自带的简单的资源管理系统，负责跟踪集群状态并调度计算任务
- Kubernetes：Google开发的一个强大的容器编排框架，用户通过Kubernetes管理容器，不需要和底层交互



目录

- 分布式计算
 - 分布式计算概述
 - 分布式计算理论基础
- 分布式系统
 - 特性
 - 类型
- 分布式计算、存储和资源管理系统
 - 案例1: Hadoop
 - 案例2: Spark
 - 案例3: 阿里开放数据处理平台



引例——问题

- 假设现有一些配置完全相同的机器
cSlave0~cSlaveN, cMaster0, cMaster1,
并且每台机器都有1个双核CPU, 5GB硬盘。
现有两个大小都是2GB的文件file0和file1
- 第一类问题：存储
 - 问题①：将file0和file1存入两台不同机器，但要求对外显示它们存于同一硬盘空间
 - 问题②：有一文件file2，大小为6GB，要求存入机器后对外显示依旧为一个完整文件
- 第二类问题：计算
 - 问题③：在问题①下，统计file0和file1这两个文件里每个单词出现的次数
- 第三类问题：可靠性
 - 问题④：假设用于解决上述问题的机器宕机了，问如何保证数据不丢失

取两台机器cSlave0和cSlave1，
cSlave0存储file0，cSlave1存储file1

将file2拆成两个大小分别为3GB的文件file2-a和file2-b，将file2-a存入cSlave0、file2-b存入cSlave1

步骤一：将cSlave1上的file1复制一份到cSlave0

网络开销

步骤二：编写一简单程序使用HashMap<String, Integer>顺序读取文件，判断新读取的单词是否存在于HashMap，存在Integer+1，不存在则HashMap里加入这个新单词，Integer置为1，记此程序为WordCount。

步骤三，将此程序WordCount放在cSlave0上执行，得出结果。

利用率低

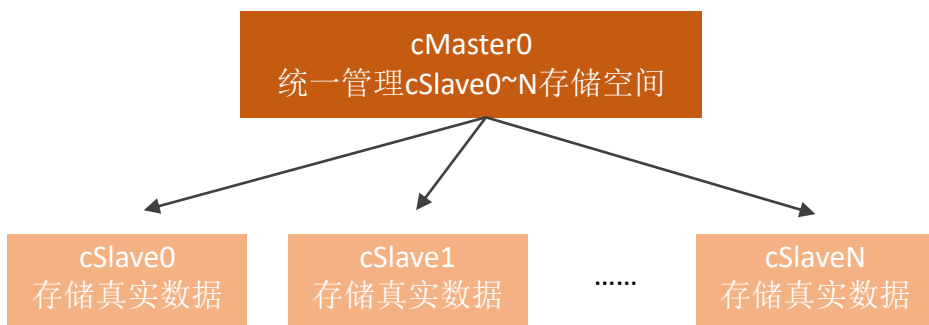
为每台机器都做磁盘冗余阵列（RAID），购买更稳定的硬件，配置最好的机房、最稳定的网络
额外的高费用



引例——问题

- 分布式存储

将多台机器硬盘以某种方式连接到一起
取机器cSlave0, cSlave1和cMaster0, 采用客户-服务器模式构建分布式存储集群
让cMaster0管理cSlave0, cSlave1。



对内 客户-服务器模式
只要保证store master正常工作，我们很容易随意添加store slave，硬盘存储空间无限大。

对外 统一存储空间，统一文件接口
整个集群就像是一台机器、一片云，硬盘显示为统一存储空间，文件接口统一。

分布式文件系统

Distributed File System, DFS

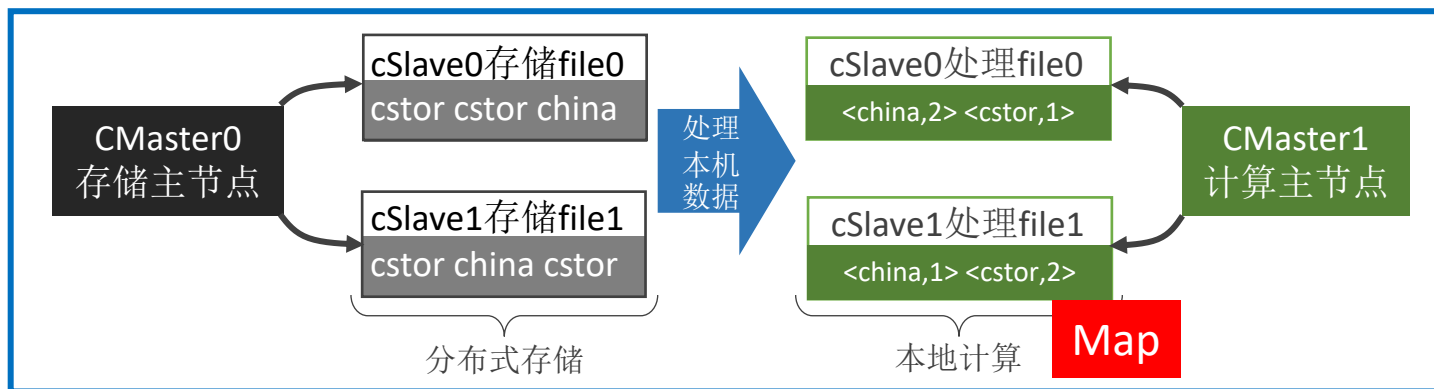
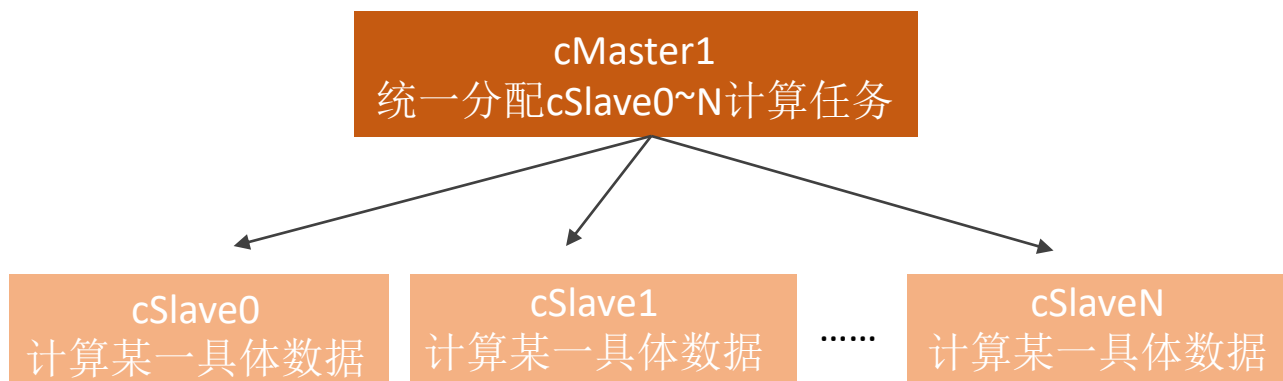


引例——问题

• 分布式计算

“移动计算比移动数据更划算”——Google论文

取新机器cMaster1作为服务器，采用客户-服务器模式构建由机器cSlave0、cSlave1和cMaster1组成的分布式计算集群。



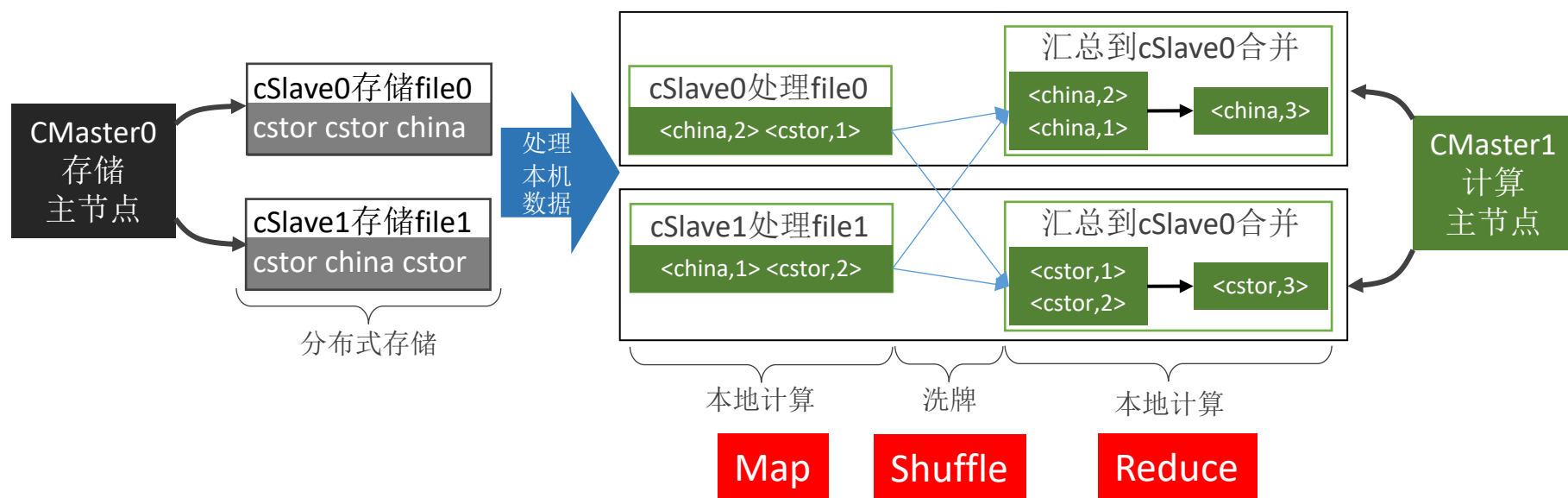
如果合并也能够并行执行呢？

Reduce



引例——问题

- 分布式计算——Hadoop MapReduce框架 大致 如下：



引例——问题

- 冗余存储与冗余计算
 - 解决可靠性问题——不单纯靠额外增加设备的备份
 - 将每台机器上存储的数据同时存于集群中的另一台机器上
 - 将每台机器上数据的计算也同时在冗余数据的机器上计算
- CMaster0明确知道每一份数据都存储在多个地方
- CMaster1会要求存有待计算数据的机器都参与计算，并选择先结束的机器计算结果
- 冗余存储不仅提高了分布式存储的可靠性，也提高了分布式计算的可靠性

可以独立存在



Hadoop——简介

- 2002
 - Apache成立开源搜索引擎项目Nutch——但开发过程中无法有效地将计算任务分配到多台机器上
 - 2004
 - 前后Google陆续发表GFS、MapReduce、BigTable
 - Apache借鉴GFS和MapReduce，实现了自己的NDFS和MapReduce
 - 2006
 - 发现Nutch侧重搜索，而NDFS和MapReduce偏向通用基础架构
 - 将NDFS和MapReduce移出Nutch，成为独立开发项目，称为Hadoop
 - Hadoop 1.0 (1.X的统称)
 - Hadoop 2.0 (2.X的统称)
- } 架构差异较大



<http://nutch.apache.org/>



Bigtable

<https://cloud.google.com/bigtable#section-10>



<http://hadoop.apache.org/>



Hadoop——简介

- Hadoop 1.0

- Hadoop Common: 支持其他两个模块的公用组件
- Hadoop DFS: 分布式文件系统
- Hadoop MapReduce: 分布式计算框架

联系HDFS和MapReduce的中间纽带:
一方面为两个组件提供公用jar包
一方面是程序员访问其他模块的接口

- Hadoop 2.0

- 分布式存储HDFS
- 分布式操作系统Yarn

- 通过调用程序库, 可使用简单的编程模型处理分布在不同机器上的大规模数据
- 采用客户-服务器模式, 很容易从一台机器扩展至成千上万台机器, 每台机器均能提供本地存储和本地计算

- 看作是Google Cloud的开源版本; 但并不拘泥于复现Google Cloud的相关产品



Hadoop——简介

- 主要应用领域
 - 构建大型分布式集群（存储+计算）
 - 最直接的应用
 - 构建大型分布式集群，提供海量存储和计算服务
 - 类似产品中国移动“大云”、淘宝“云梯”
 - 数据仓库（存储）
 - 存储半结构化业务数据，通过Hive、Hbase提供报表查询之类服务
 - 数据挖掘（计算）
 - 大数据环境下的数据挖掘思路和算法没有太大变化
 - 硬盘性能和内存大小带来的限制——通过分布式集群解决硬件限制



Hadoop——2.0 体系架构

- 公共组件Common
- 分布式文件系统HDFS
- 分布式操作系统Yarn（分布式资源管理系统）
- MapReduce引擎
- Common
 - 定位：其他模块的公共组件，定义程序员取得集群服务的编程接口，为其他模块提供共用API
 - 作用：降低Hadoop设计的复杂性，减少其他模块间耦合性，增强Hadoop健壮性
 - 功能
 - 提供公用API和程序员编程接口（例如Configuration类）
 - 本地Hadoop库（例如压缩解压缩用的是Hadoop本地库）
 - 超级用户superuser（类似sudo -u username）
 - 服务级别认证
 - HTTP认证



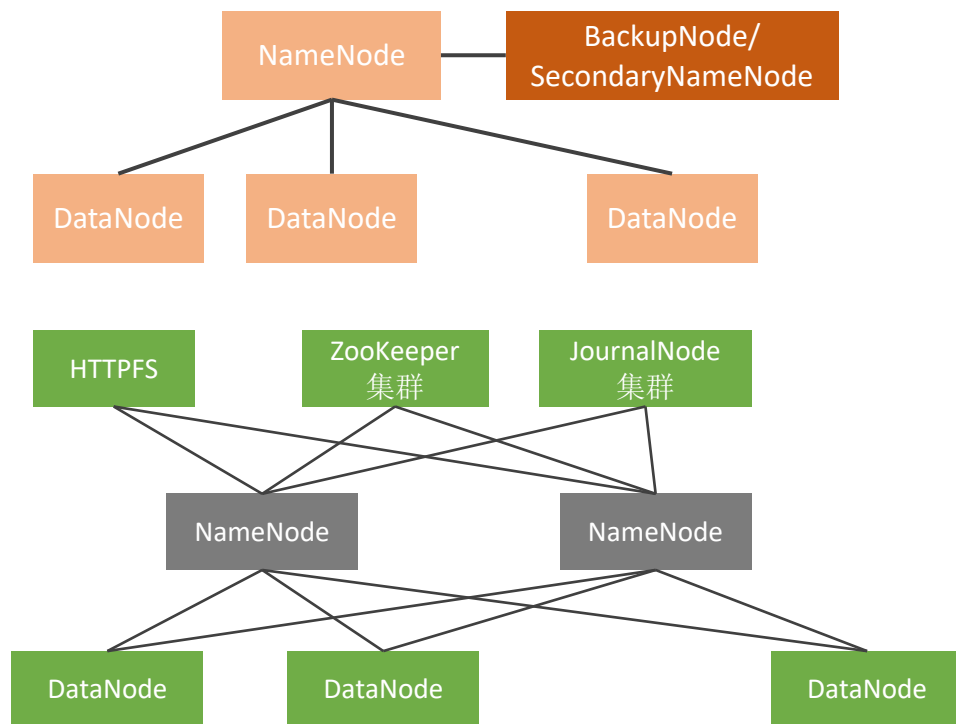
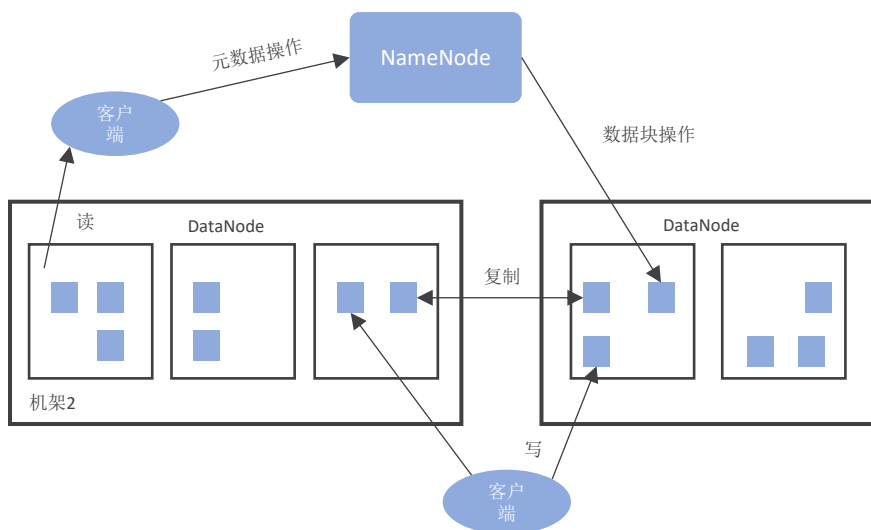
Hadoop——2.0——HDFS

- 定位

- 高容错、高扩展、高可靠，并提供服务访问接口，如API接口和管理员接口

- 体系架构

- 架构：master/slave；文件分块存储；namenode/datanode；
- 典型拓扑：一般/商用（ZooKeeper选举ActiveNamenode，JourNalNode两个Namenode交换数据）



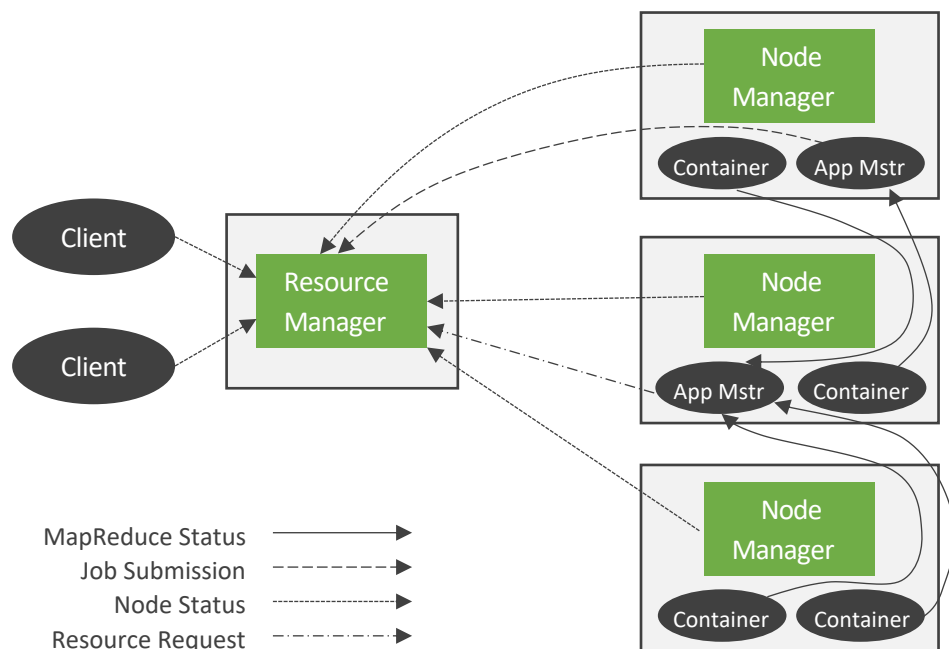
Hadoop——2.0——HDFS

- 内部特性
 - 冗余备份、副本存放、副本选择、心跳检测
 - 数据完整性检测、元数据磁盘失效
 - 简单一致性模型、流式数据访问、客户端缓存
 - 流水线复制、架构特征、超大规模数据集
- 对外功能
 - Namenode高可靠性：配置多个NameNode，一个失效时立即替换
 - HDFS快照：当数据损坏时，支持回滚到正确的时间节点
 - 元数据管理与恢复工具：通过命令hdfs oiv和hdfs oev管理修复fsimage和edits
 - HDFS安全性：用户和文件级别安全认证、机器和服务级别安全认证
 - HDFS配额功能：管理目录或文件配额大小
 - HDFS C语言接口：使用C语言操作HDFS的接口
 - HDFS Short-Circuit功能：客户端可以绕过Datanode直接读取本机数据，加快map操作
 - WebHdfs：通过Web方式操作HDFS（插、删、改、查）



Hadoop——2.0——Yarn

- 资源管理——Yarn：管理计算机资源、提供用户和程序访问系统资源的API



Yarn在执行时包含以下独立实体：

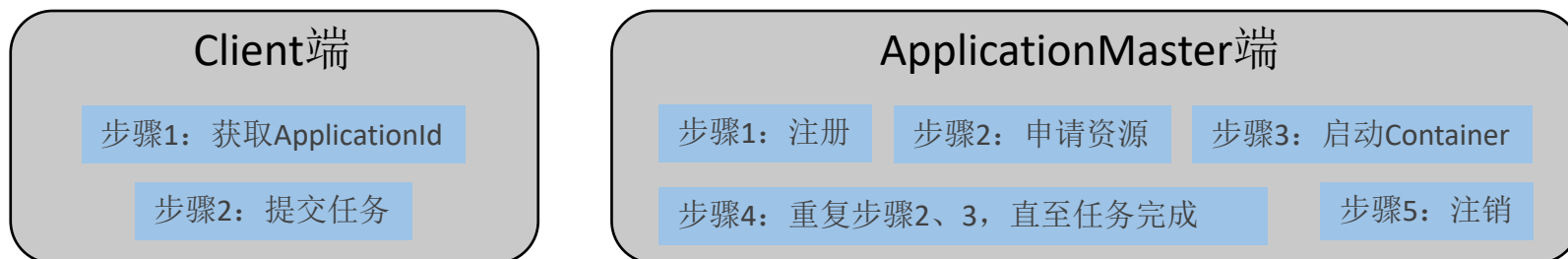
- ① Client：客户端，负责向集群提交作业。
- ② ResourceManager：集群主进程，仲裁中心，负责集群资源管理和任务调度。
- ③ Scheduler：资源仲裁模块。
- ④ ApplicationManager：选定，启动和监管ApplicationMaster。
- ⑤ NodeManager：集群从进程，管理监视Containers，执行具体任务。
- ⑥ Container：本机资源集合体，如某Container为4个CPU，8GB内存。
- ⑦ ApplicationMaster：任务执行和监管中心。

- 任务调度策略是Yarn的核心问题，ResourceManager的Scheduler模块支持插拔，通过配置文件，用户可以个性化指定其调度策略。自带两种策略
 - 容量调度算法CapacityScheduler：按照配置好的资源配比为不同层级的用户分配最大可用资源
 - 公平调度算法FairScheduler：任务公平使用集群资源，短任务在合理时间内完成；长任务不至于永远等待



Hadoop——2.0——Yarn

- 资源管理——Yarn是可编程的，不仅仅支持自带的MapReduce，还可以自定义算子
- 一套编程协议
 - Client负责提交任务，ApplicationMaster负责执行任务
 - Client中与RM通信；ApplicationMaster与RM通信；ApplicationMaster与NM通信
 - 编写符合协议的Client和ApplicationMaster即可



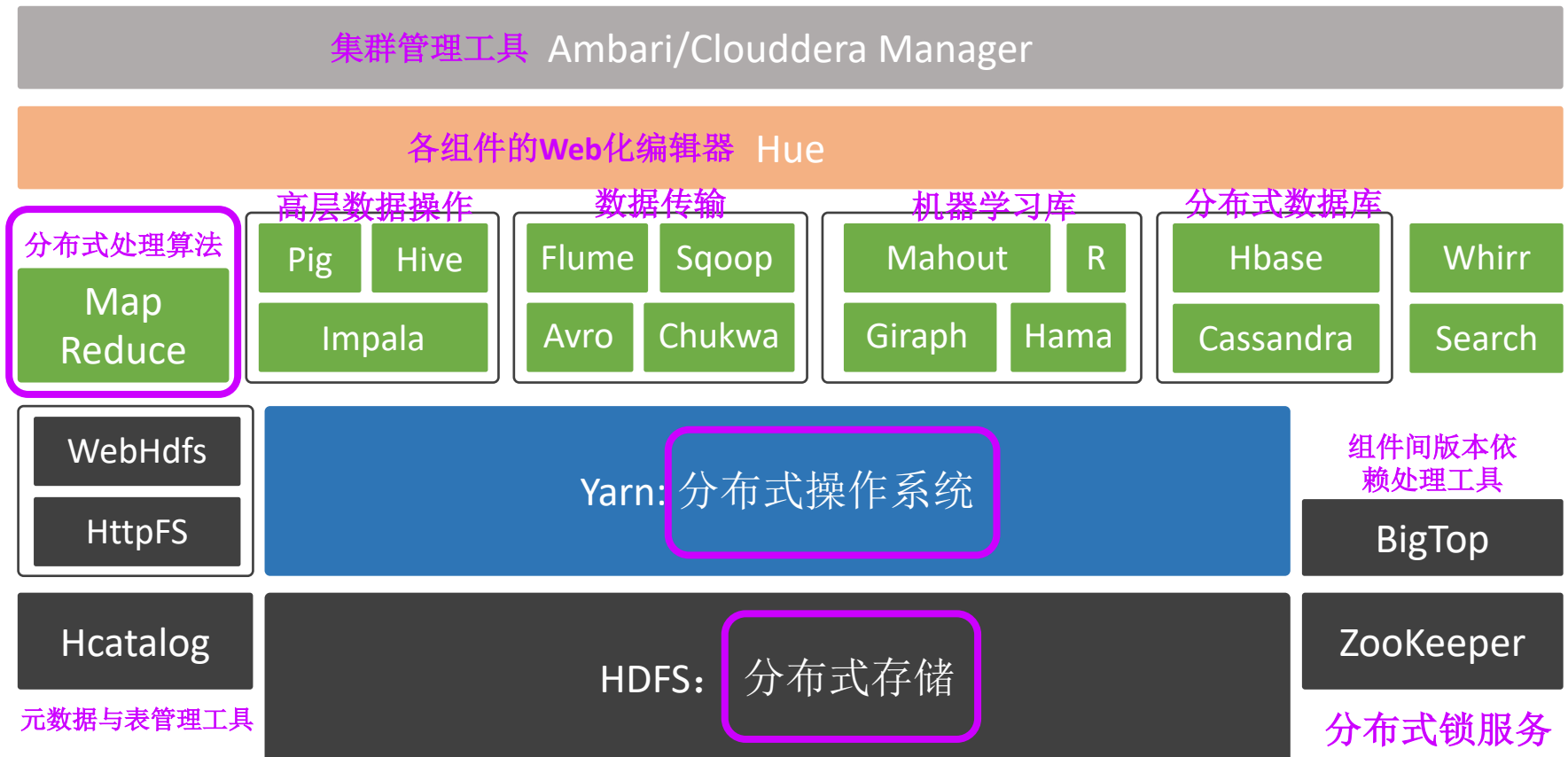
- Hadoop默认实现了MapReduce的Client和ApplicationMaster
 - MRClientService
 - MRAppMaster
 - Yarn处理MR程序时使用了各种默认的种类

Scheduler也是可编程的

所以我们只是写写Map函数、写写Reduce函数

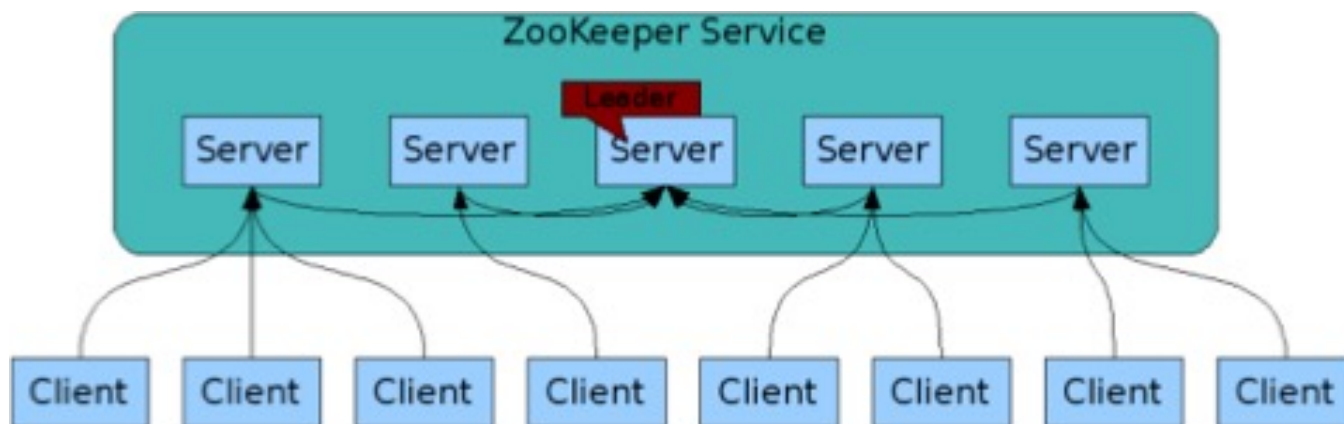


Hadoop——2.0 大家族



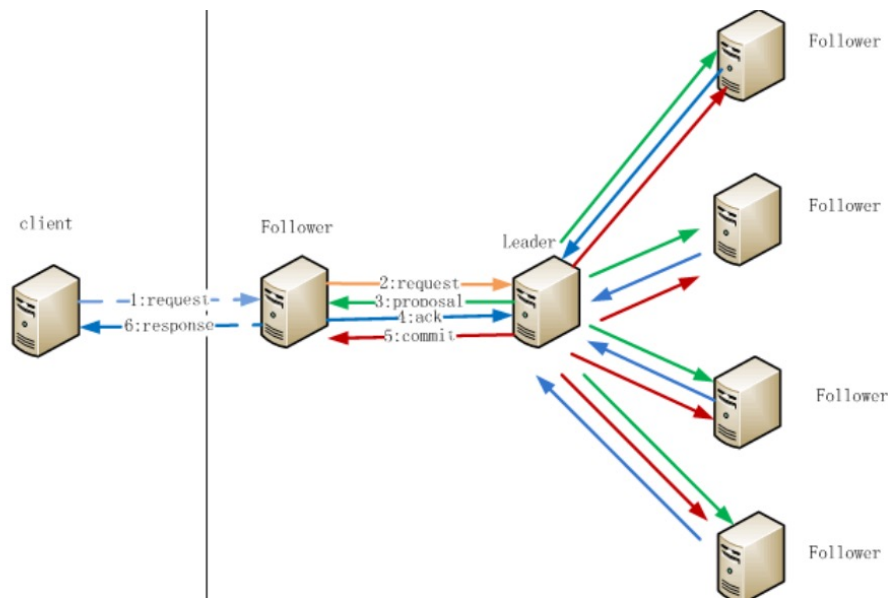
Hadoop—大家族—Zookeeper设计目标与读写机制

- 目的
 - 大部分分布式应用需要一个主控、协调器或控制器来管理物理分布的子进程（如资源、任务分配等）
 - 缺乏通用的协调程序，需要各自编写
- 目标
 - 提供通用的分布式协调服务
- 由多个Server组成的集群；一个Leader，多个Follower
 - 每个Server都保存了一份数据副本
 - 全局数据一致；分布式读写
 - 更新请求转发，由Leader实施



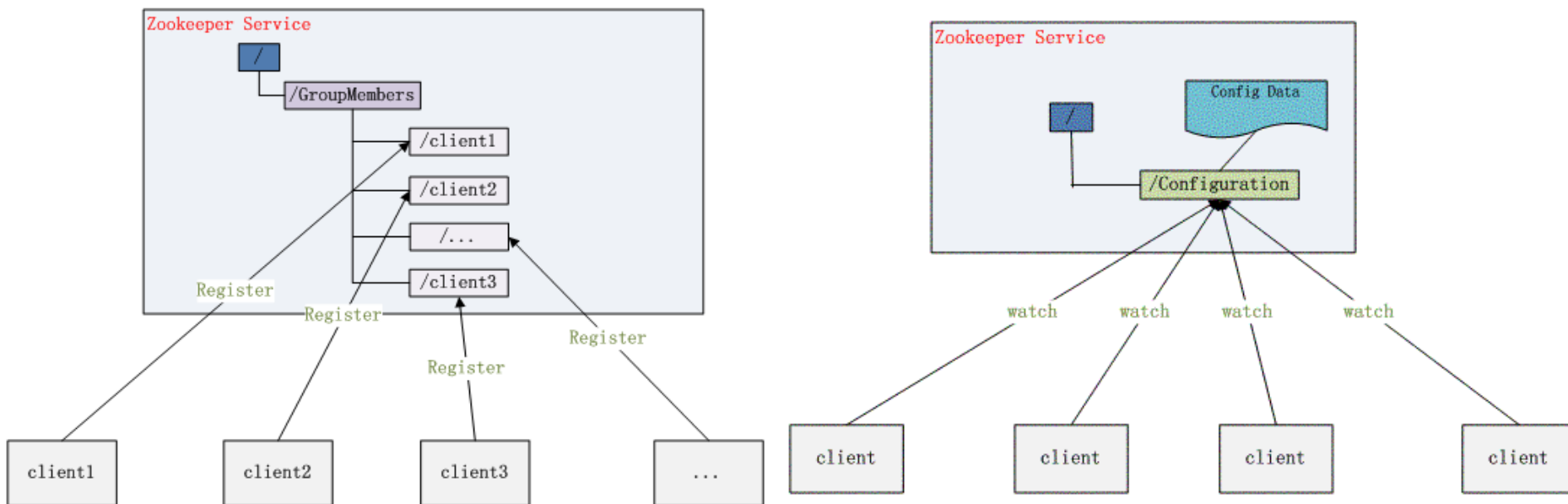
Hadoop—大家族—Zookeeper的使用约定

- 更新请求顺序执行
 - 来自同一个Client的更新请求按其发送顺序依次执行
- 数据更新原子性
 - 一次数据更新要么成功，要么失败。不存在部分数据写入成功或失败的情况
- 全局唯一数据视图
 - Client无论连接哪个Server，数据视图都是一致的
- 实时性
 - 在一定时间范围内，Client能读到最新数据



Hadoop—大家族—Zookeeper其它使用场景

- 统一命名服务 Name Service
 - 为集群产生一系列不重复的名称，随便关联或不关联资源；就像JNDI，就像生成数据库表的主键
- 配置管理 Configuration Management
 - 保存所有机器都相同的配置信息，一旦改变，则通知所有机器更新



Hadoop—大家族—Hbase

- 基于Hadoop的开源分布式数据库，以Google BigTable为原型

- 高可靠性
- 高性能
- 列存储
- 可伸缩
- 实时读写

行键	时间戳	列族contents	列族anchor	列族mime
"com.cnn.www"	t9		anchor:cnnsi.com= "CNN"	
	t8		anchor:my.look.ca= "CNN.com"	
	t6	contents:html="<html>..."		mime:type="text/html"
	t5	contents:html="<html>..."		
	t6	contents:html="<html>..."		

- 逻辑模型：用户对数据的组织形式

- 行键、列（<列族>:<限定符>）、时间戳；字节码，无类型

- 物理模型：在设备上具体存储形式

- 将行按照列族分割存储；逻辑空值无存储

行键	时间戳	列族contents
"com.cnn.www"	t6	contents:html="<html>..."
	t5	contents:html="<html>..."
	t3	contents:html="<html>..."
行键	时间戳	列族anchor
"com.cnn.www"	t9	anchor:cnnsi.com= "CNN"
	t8	anchor:my.look.ca= "CNN.com"
行键	时间戳	列族mime
"com.cnn.www"	t6	mime:type="text/html"

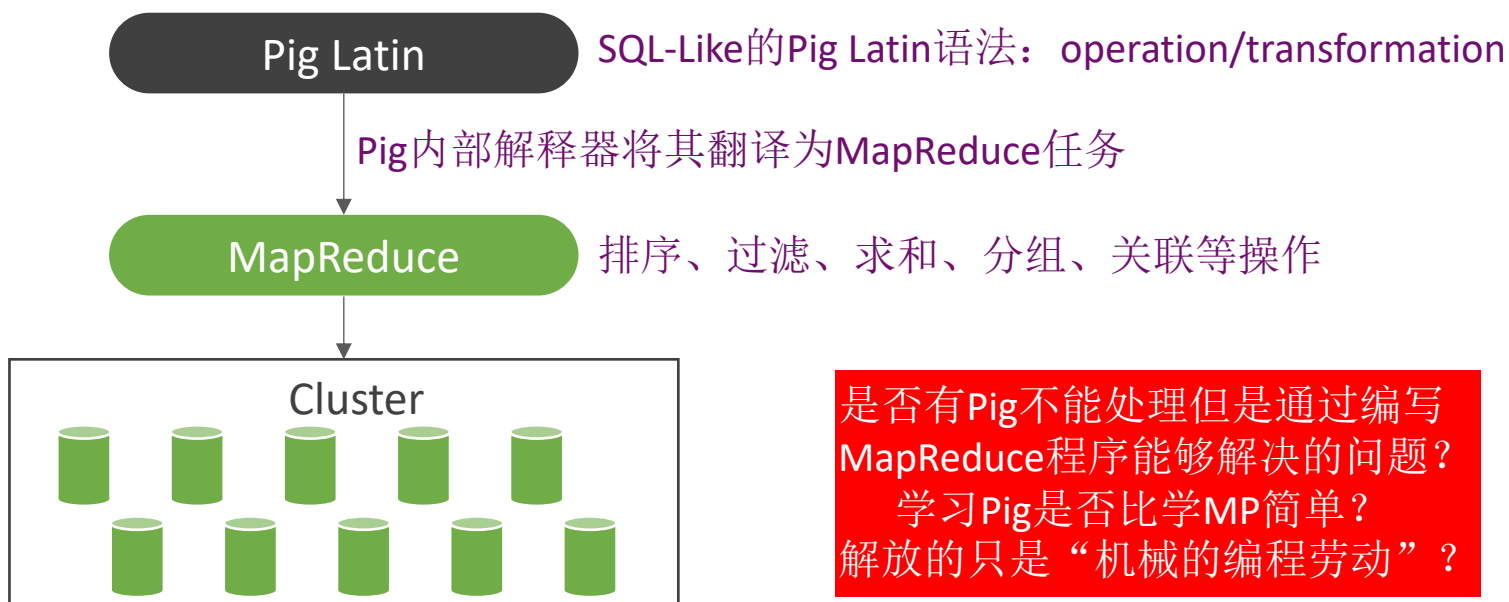
- 架构

- HMaster, HRegionServer
- ZooKeeper保证HMaster无单点故障，并为HMaster管理HRegionServer集群



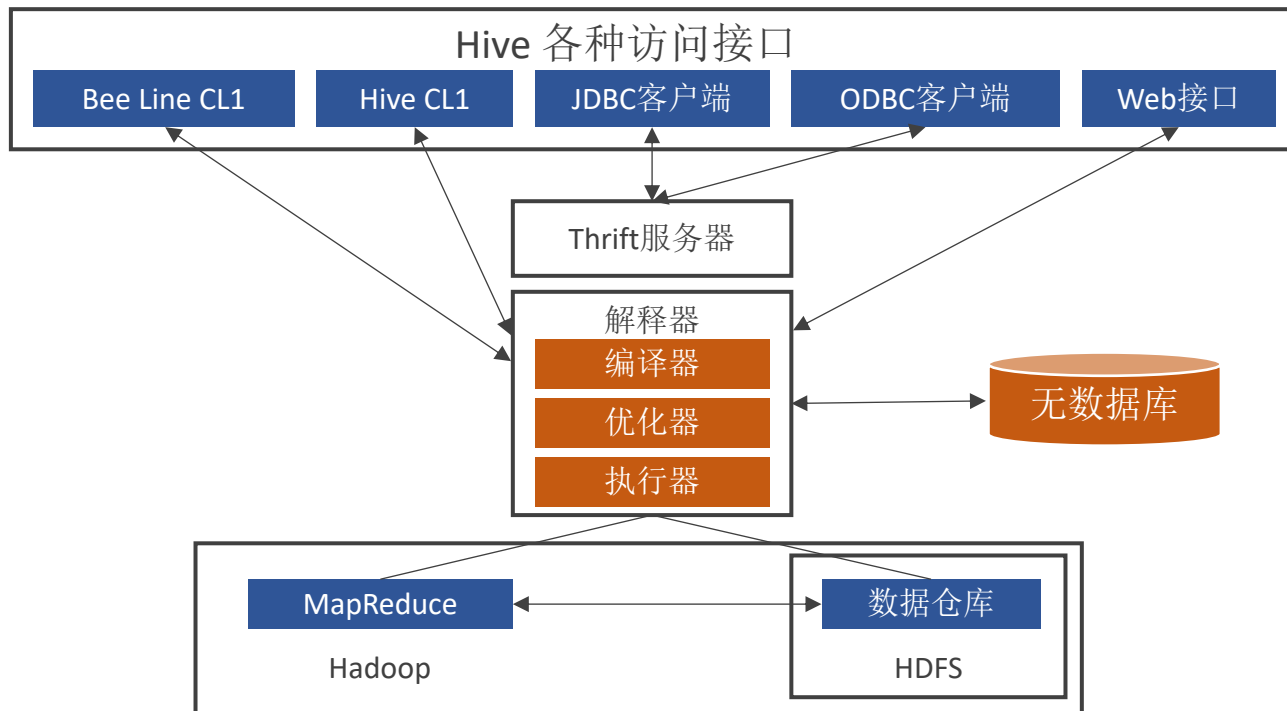
Hadoop—大家族—Pig

- 构建在Hadoop之上，用来处理大规模数据集的脚本语言平台
- 功能
 - 将根据业务逻辑写好的数据流脚本翻译成多个HDFS、Map和Reduce的操作
 - 为Hadoop提供更高层次的抽象，将程序员从具体编程中解放出来



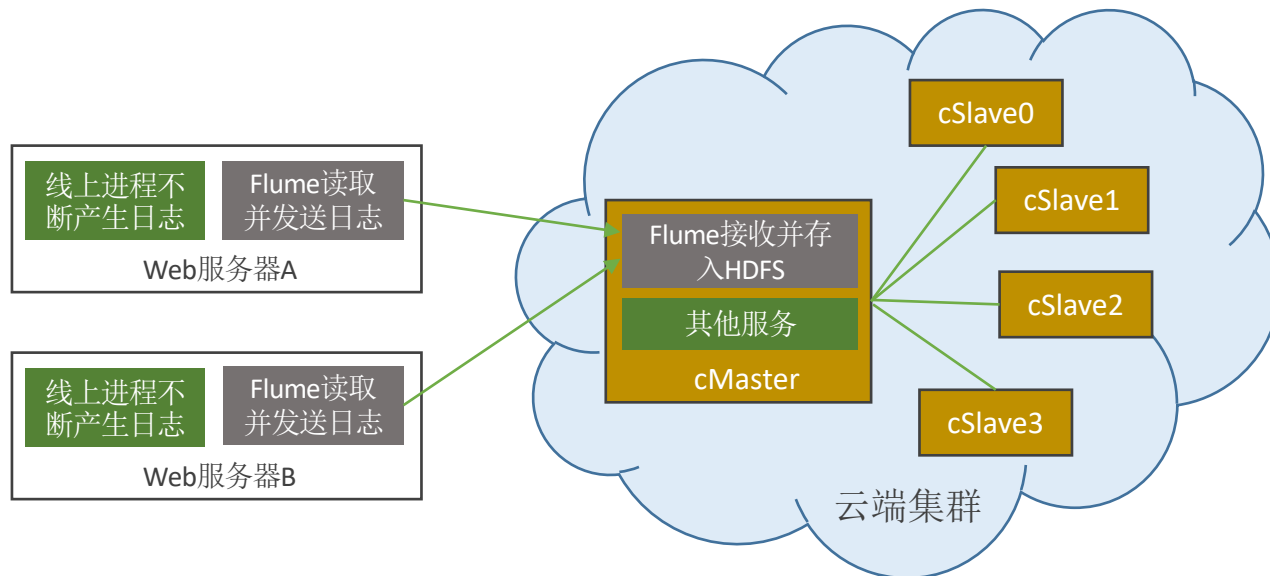
Hadoop—大家族—Hive

- 数据仓库
- 和Pig很像，也是对MapReduce的一个高层抽象
- 区别
 - Hive更像是一个数据库，使用类SQL的HSQL提供查询和操作服务
 - Pig更像是一个MapReduce的另一个编程入口，更多地利用Pig编写程序



Hadoop—大家族—Flume

- 分布式高性能、高可靠地数据传输工具
 - 智能路由器，强大地分用、复用、断网续存功能
- 典型场景：众多生产机器的日志数据实时导入HDFS



目录

- 分布式计算
 - 分布式计算概述
 - 分布式计算理论基础
- 分布式系统
 - 特性
 - 类型
- 分布式计算、存储和资源管理系统
 - 案例1: Hadoop
 - 案例2: Spark
 - 案例3: 阿里开放数据处理平台



Spark——简介

- 当今大数据领域最活跃、最热门的大数据计算处理框架

<https://amplab.cs.berkeley.edu/about/>

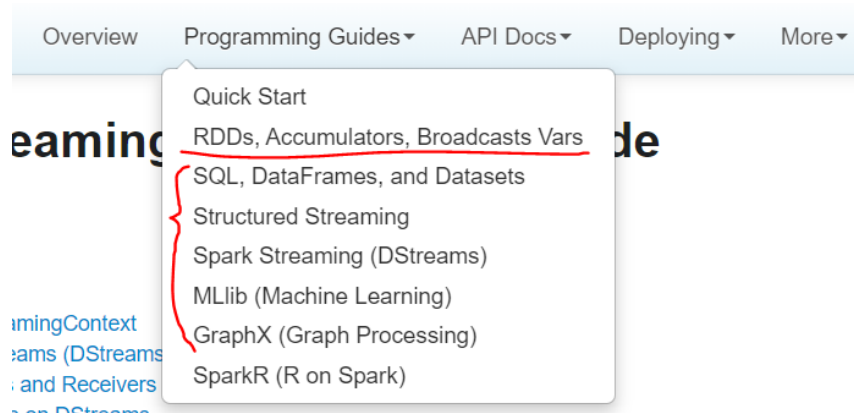
- 2009年——诞生于美国加州大学伯克利分校AMP实验室
- 2013年——Spark成为Apache基金项目
- 2014年——成为Apache基金顶级项目

解决批处理问题
Hadoop(2006)上的应用程序
DAG描述的任务流处理

- 一体化、多元化的大数据处理体系

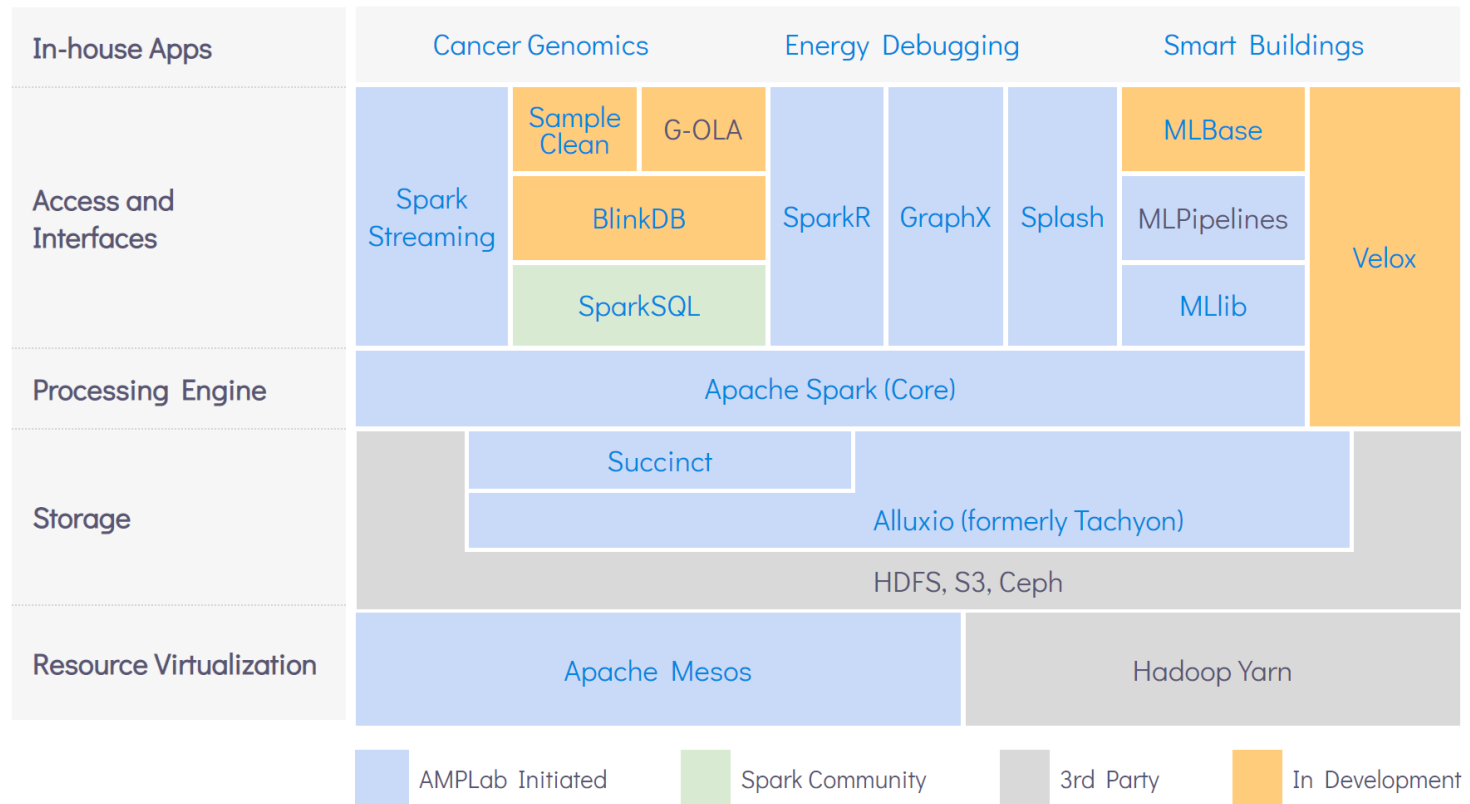
- Spark SQL
- Spark Streaming
- Spark Mllib
- Spark GraphX

Batch Processing
Stream Processing
Adhoc Query



Spark——简介

- 由Spark在伯克利的数据分析软件栈BDAS (Berkeley Data Analytics Stack) 中的位置可见, Spark专注于数据的计算, 而数据的存储在生产环境中往往还是由Hadoop分布式文件系统HDFS承担。



BDAS Components

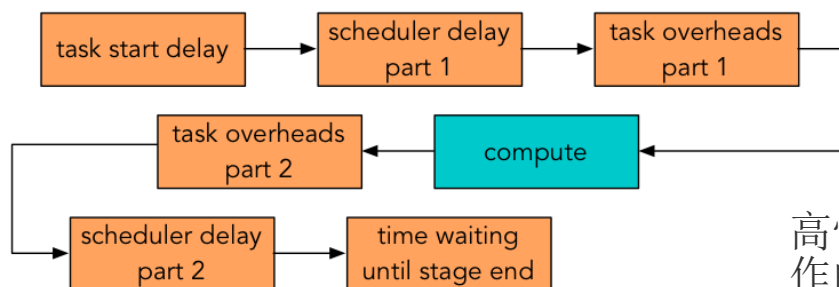
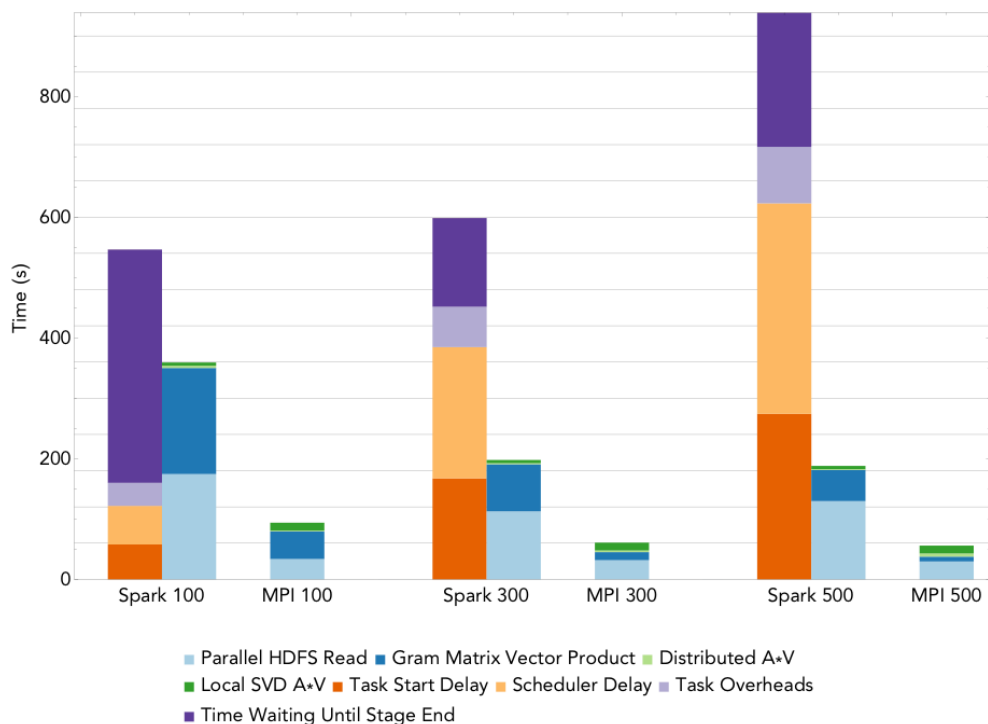


Spark——简介

- Spark MLlib和基于MPI(Message passing interface)的机器学习算法实现对比

Science Area	Format/Files	Dimensions	Size
MSI	Parquet/2880	8,258,911 × 131,048	1.1TB
Daya Bay	HDF5/1	1,099,413,914 × 192	1.6TB
Ocean	HDF5/1	6,349,676 × 46,715	2.2TB
Atmosphere	HDF5/1	26,542,080 × 81,600	16TB

	Nodes / cores	MPI Time	Spark Time	Gap
NMF	50 / 1,600	1 min 6 s	4 min 38 s	4.2x
	100 / 3,200	45 s	3 min 27 s	4.6x
	300 / 9,600	30 s	70 s	2.3x
PCA (2.2TB)	100 / 3,200	1 min 34 s	15 min 34 s	9.9x
	300 / 9,600	1 min	13 min 47 s	13.8x
	500 / 16,000	56 s	19 min 20 s	20.7x
PCA (16TB)	MPI: 1,600 / 51,200 Spark: 1,522 / 48,704	2 min 40 s	69 min 35 s	26x



高性能使用方式即是通过编程，使得作业具有多节点协作的能力，而多个节点之间的协作是通过**MPI**来实现的。

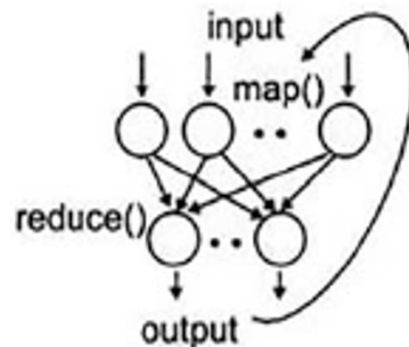
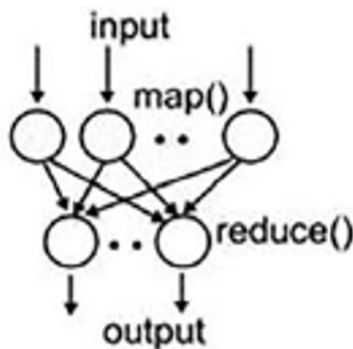
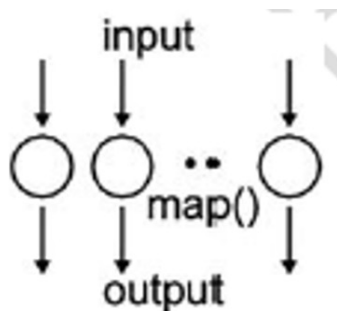
<https://computing.llnl.gov/tutorials/mpi/>



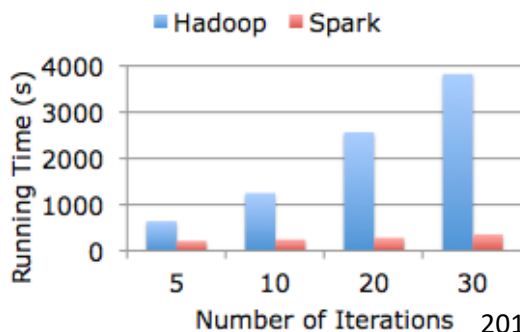
2016年6月

Spark——相比于Hadoop的优势

- Hadoop到底哪里不好了? !!
- 支持多种Map-Reduce计算模式



- 说来说去就是Map和Reduce，有没有点其他花样？
 - 计算需求可不止Map和Reduce
 - 没算两下就要我写硬盘，我不累，硬盘也累了
- 支持多种数据计算需求；
- 不写硬盘不写硬盘不写硬盘



基于内存的计算范式

流式迭代的类MR计算

图数据结构的计算

通用并行计算



Spark——相比于 ~~Hadoop~~ 的优势 MapReduce

- Spark曾经是一个Hadoop应用程序
 - Yarn的可扩展特性
 - Spark是在Hadoop开创的分布式计算框架下对MapReduce编程范式进行扩展的一种更加通用的并行计算框架
 - 独立性更强
 - 基于内存：RDD，速度更快；基于内存的计算快100x倍，基于硬盘的快10x倍 
 - 支持更多数据计算方法：transformation, action
1. 易用
支持Scala、java、python和R等多种编程语言
 2. 强大
支持SQL、Streaming、Graph和Machine Learning等多种应用场景
 3. 通用
适用于自带的Standalone、Mesos、Yarn等多种不同分布式集群管理框架；
适用多种不同数据存储方式（数据读取接口丰富）



Spark——架构 or 生态

应用开发
工具包



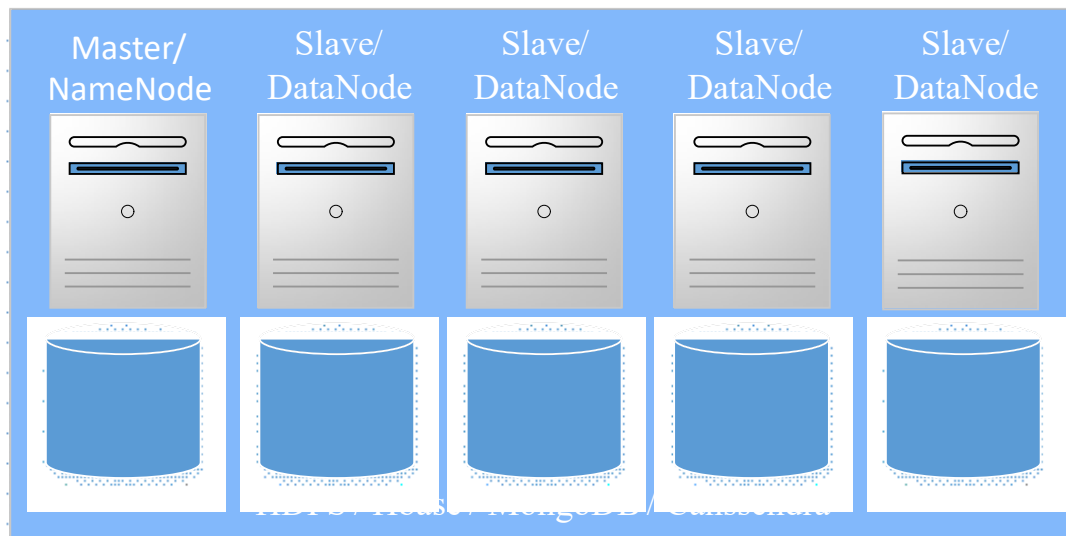
计算平台



集群管理
方式



数据存储
方式



Spark——架构 or 生态——高层库

- Spark SQL

- 前身是Shark：基于Hive的Spark SQL，代码量大、复杂，难优化和维护
- 交互式查询、标准访问接口、兼容Hive
- 专门用于处理结构化数据：
- 分布式SQL引擎；在Spark程序中调用API

- Spark Streaming

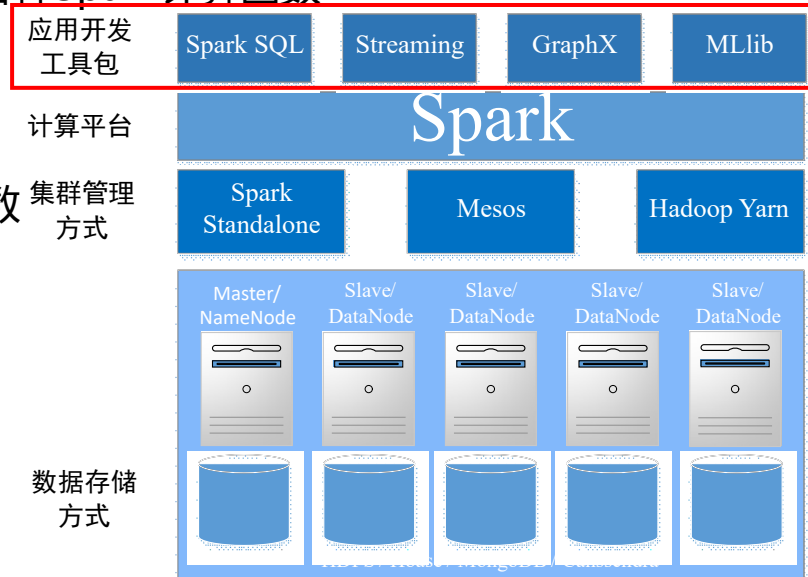
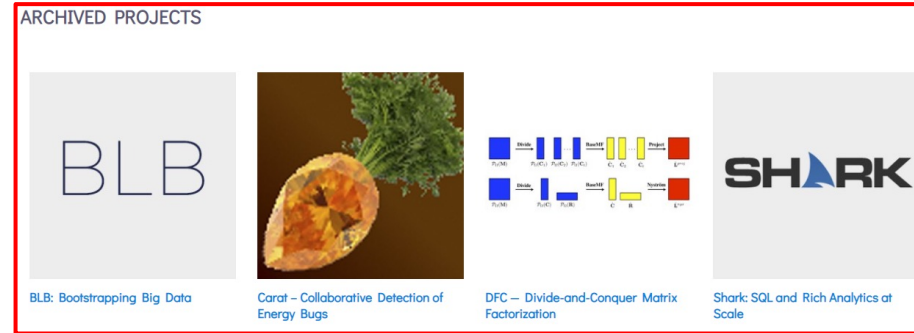
- 实时对大量数据进行快速处理，处理周期短
- 对数据分段、定义了自动监听更新的框架、提供各种Spark计算函数

- Spark GraphX

- 以图为基础数据结构的算法实现和相关应用
- 使用RDD存储图节点和边信息，提供各种计算函数

- Spark MLlib

- 为解决机器学习开发的库
- 包括分类、回归、聚类和协同过滤等



Spark——架构 or 生态——集群管理

- Standalone mode

- 原生集群管理功能：任务调度、资源分配等

- Hadoop Yarn

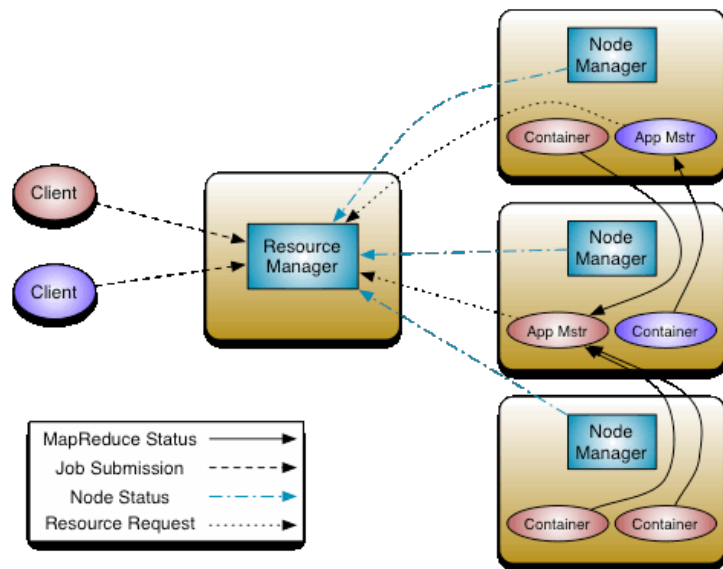
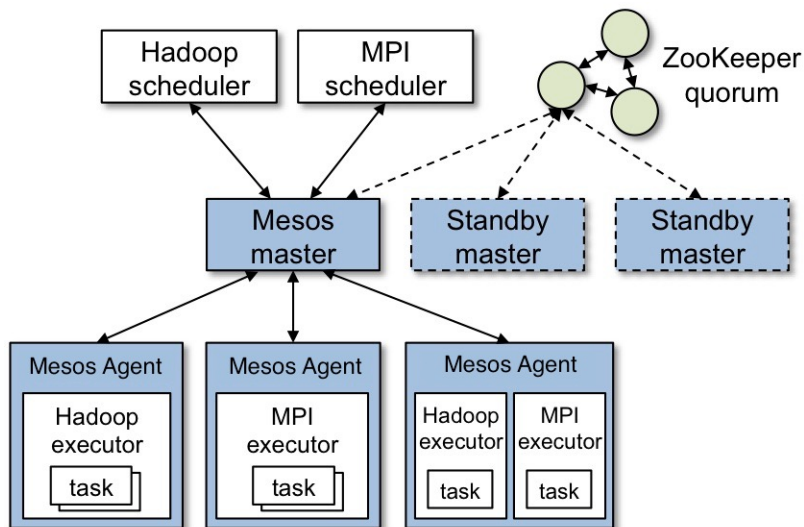
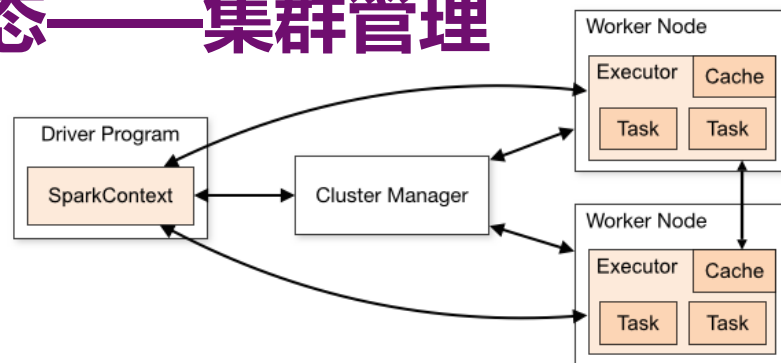
- Hadoop MapReduce的第二个版本架构，把资源管理和任务管理剥离开；实现了静态资源分配和动态资源分配功能；

- Apache Mesos



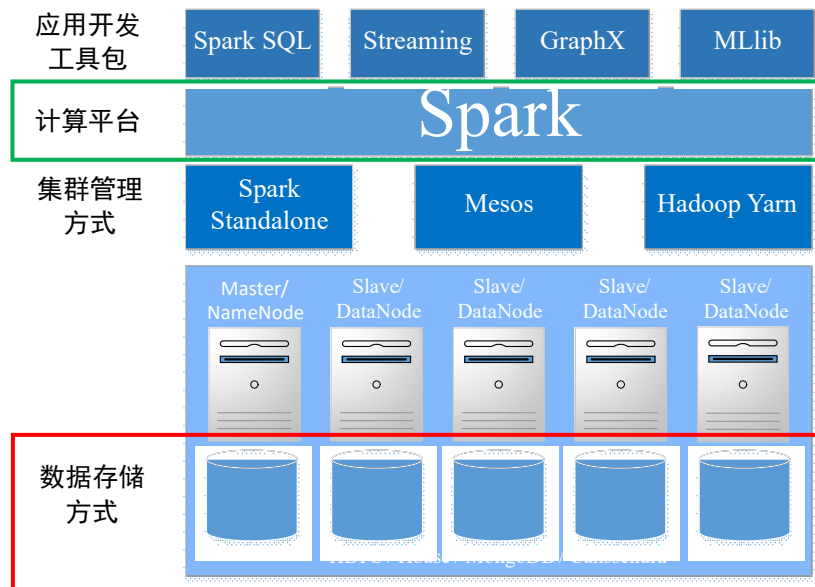
<http://mesos.apache.org/>

- 从分布式计算节点上抽象CPU, memory, storage, and other compute resources给其他框架使用，实现了静态资源分配功能



Spark——架构 or 生态——数据存储

- HDFS
 - Hadoop分布式文件系统
- Hbase
 - 基于HDFS的非关系型数据库(NoSQL数据库)
- MongoDB
 - 基于分布式存储的数据库
 - 介于关系型和非关系型数据库之间
 - 是NoSQL数据库中最像关系型数据库的一个
 - 功能非常强大：支持多种开发语言、支持完全索引、支持查询等
- Cassandra
 - 基于列的分布式数据库，易扩展、模式灵活、按照范围查询等
- 它们和Spark的关系：Spark是一个计算程序，需要读取数据和存储数据，它们就是数据存储的地方



云数据库 GaussDB(for Cassandra)

GaussDB(for Cassandra) 是一款基于华为自研的计算存储分离架构的分布式数据库，兼容 Cassandra生态的云原生NoSQL数据库，支持类SQL语法CQL。在华为云高性能、高可用、高可靠、高安全、可弹性伸缩的基础上，提供了一键部署、快速备份恢复、计算存储独立扩容、监控告警等服务能力。

立即购买

价格计算器

帮助文档

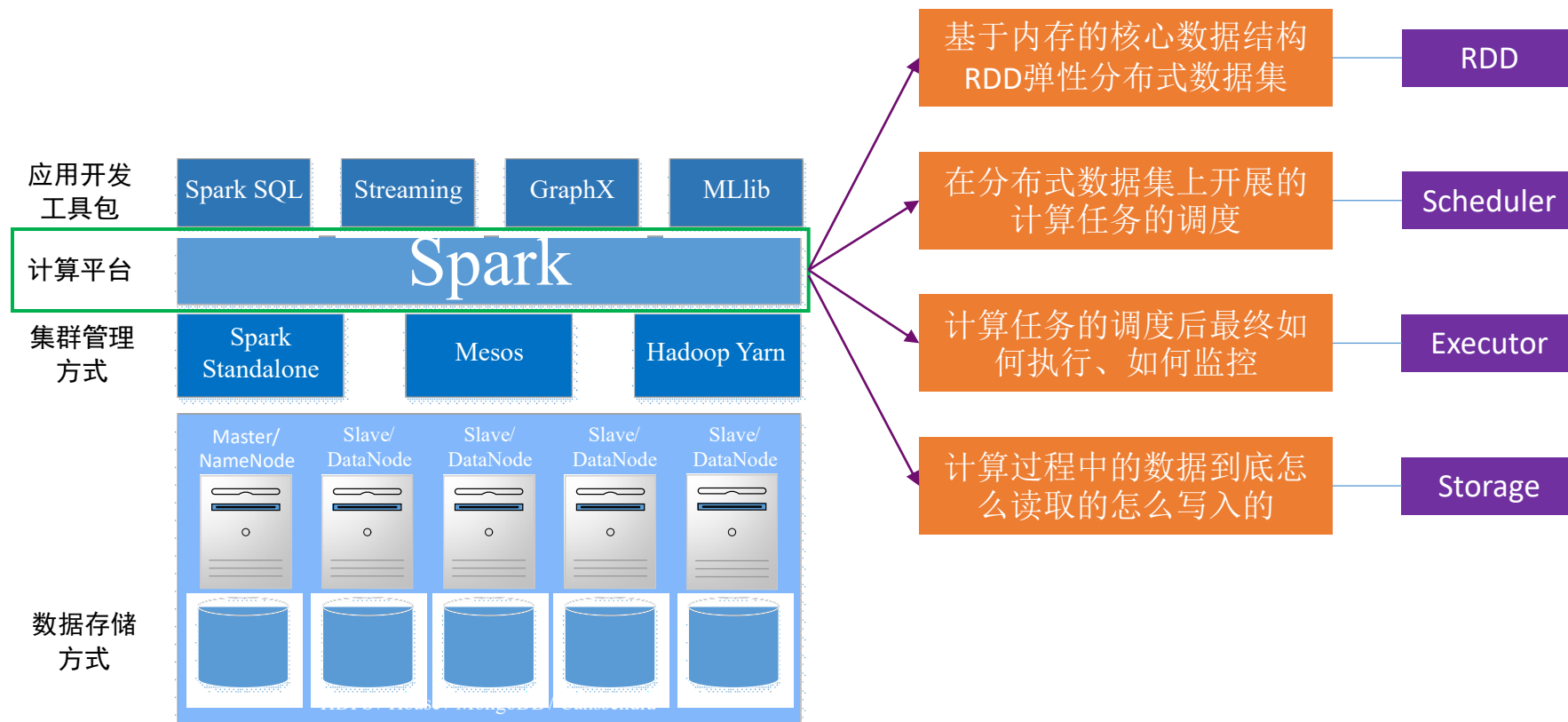


Spark——架构 or 生态——部署模式

- Spark框架的部署：Local模式和Cluster模式
 - Local是方便初学者入门学习和测试用的部署模式
 - Cluster是真正的集群模式
 - Standalone集群管理器
 - Yarn集群管理器
 - Mesos集群管理器
- Spark应用程序的提交模式：Client模式和Cluster模式
 - Local部署模式只支持Client提交模式
 - 真正的Cluster集群部署模式同时支持Client和Cluster提交模式
- Client提交模式
 - 在worker节点启动Driver程序运行应用程序，结果返回到Client端
- Cluster提交模式
 - 在Master上启动Driver程序，结果不会返回给client而是保存在Master上



Spark——内核机制解析



Spark——内核机制解析——RDD概念

- Spark是建立在RDD(Resilient Distributed Datasets, 弹性分布式数据集)之上的
- RDD使得Spark可以用一致的方式处理大数据的不同应用场景

- RDD是一个容错的、并行的逻辑数据结构

- 数据存储到内存和磁盘中
- 控制数据分区
- 丰富的API操作数据

高度受限的共享内存模型
“只读”

1. 从外界读入, 2. 由其他RDD产生

创建RDD的一系列转换被记录下来
精确到具体分区的数据恢复

- 解决的具体问题——不能基于内存共享数据, 反复读写磁盘
 - Hadoop的MapReduce对迭代式算法支持的效率不高, 图算法和机器学习算法
 - 交互式数据挖掘工具中反复查询一个数据子集

逻辑数据块**Partition**, 对应的物理存储单元**Block**
由**BlockManager**管理物理分区
(画图稍微解释一下吧)



Spark——内核机制解析——RDD定义

- RDD的五大特性

- 分区列表：记录了数据块所在的分区位置；一个RDD对应的数据是切分为数据块存储在集群的不同节点上的

```
@transient def getPartitions_ : Array[Partition] = null
```

- 依赖列表：记录了当前这个RDD依赖于哪些其它的RDD

```
private var dependencies_ : Seq[Dependency[_]] = null
```

依赖关系类型

- 计算函数compute，用于计算RDD各个分区的值

```
def compute(split: Partition, context: TaskContext): Iterator[T]
```

- 可选：分区器，子类可以重新指定新的分区方式：Hash和Range

```
@transient val partitioner: Option[Partitioner] = None
```

- 可选：计算各分区时优先的位置列表。例如从HDFS文件生成RDD时，HDFS文件所在位置就是对应生成的RDD分区所在位置的优先选择

```
protected def getPreferredLocations(split: Partitiion): Seq[String] = Nil
```

宽依赖/窄依赖 → Shuffle:HushShuffle/SortShuffle

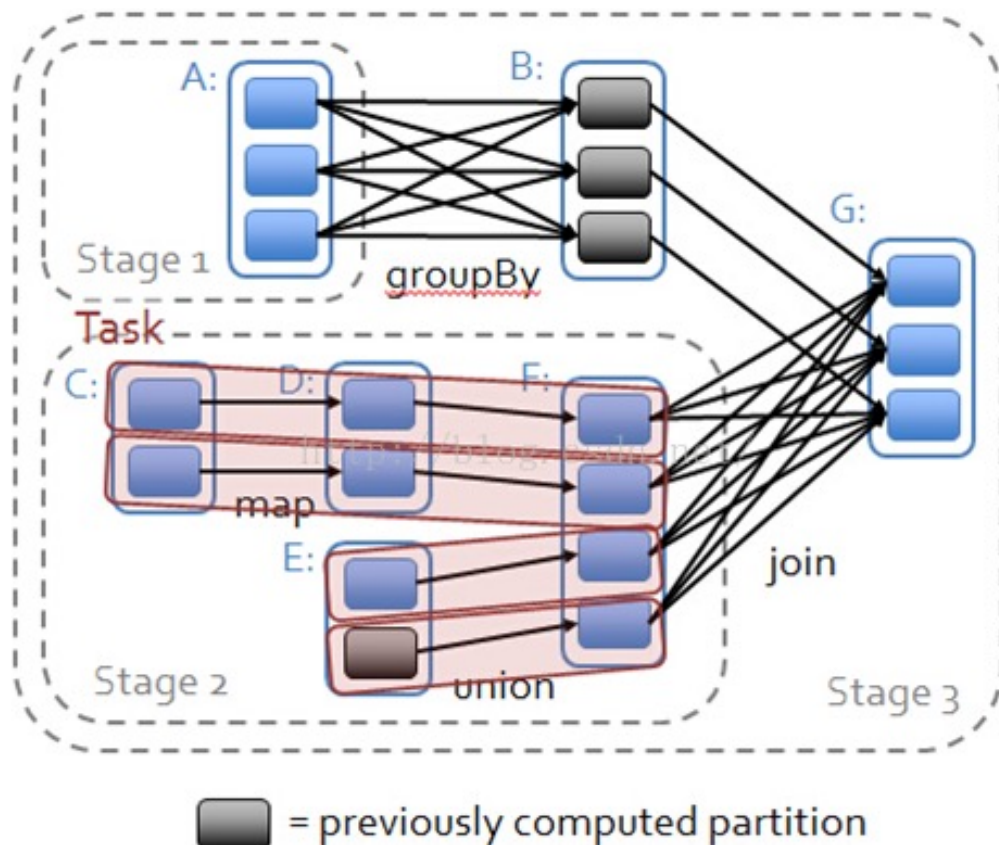
➡ Shuffle RDD/Operation

➡ Non Shuffle RDD/Operation



Spark——内核机制解析——基于RDD的计算

- 基于RDD的计算过程及相关术语



Transformation

Action

Task

Stage

Job

Application

Shuffle Dependency

Narrow Dependency

Directed Acyclic Graph(DAG)

Spark——内核机制解析——RDD弹性体现

- RDD弹性的7个方面

- 自动进行内存和磁盘存储的切换

- 优先内存，实在放不下则放到磁盘里

效率考虑

- 基于Lineage(血统)的高效容错机制

- 记录每一个数据分片的计算来源，便于快速恢复

效率考虑

容错考虑

- Task如果失败会自动进行特定次数的重试

- TaskScheduler获取一个Stage的TaskSet，运行它们；默认4次

容错考虑

- Stage如果失败会自动进行特定次数的重试

- DAGScheduler调度Stage，Stage跟踪执行情况；默认4次

容错考虑

- Checkpoint和Persist(检查点和持久化)可主动或被动触发

- 用户能见的RDD可以主动调用，自动产生的中间RDD则可配置

效率考虑

容错考虑

- 数据调度弹性（DAGScheduler、TaskScheduler和资源管理无关）

- 任务调度、数据分配和计算资源的管理解耦：Yarn管理Spark集群

效率考虑

- 数据分片的高度弹性：合并分片和切分分片

- 根据产生的分片（数据块）大小自动选择继续切分还是放到磁盘

效率考虑



Spark——内核机制解析——RDD创建

- RDD的创建方式

- 通过已存在的数据集合创建（变量）

- parallelize(...)

```
def parallelize[T: ClassTag](  
  seq: Seq[T],  
  numSlices: Int = defaultParallelism): RDD[T] = withScope {  
  assertNotStopped()  
  new ParallelCollectionRDD[T](this, seq, numSlices, Map[Int, Seq[String]]())  
}
```

- 通过HDFS和本地文件系统创建

- textFile(...)

```
def textFile(  
  path: String,  
  minPartitions: Int = defaultMinPartitions): RDD[String] = withScope {  
  assertNotStopped()  
  hadoopFile(path, classOf[TextInputFormat], classOf[LongWritable], classOf[Text],  
    minPartitions).map(pair => pair._2.toString).setName(path)  
}
```

- 通过其他RDD转换

- transformation操作

- 从数据库读入

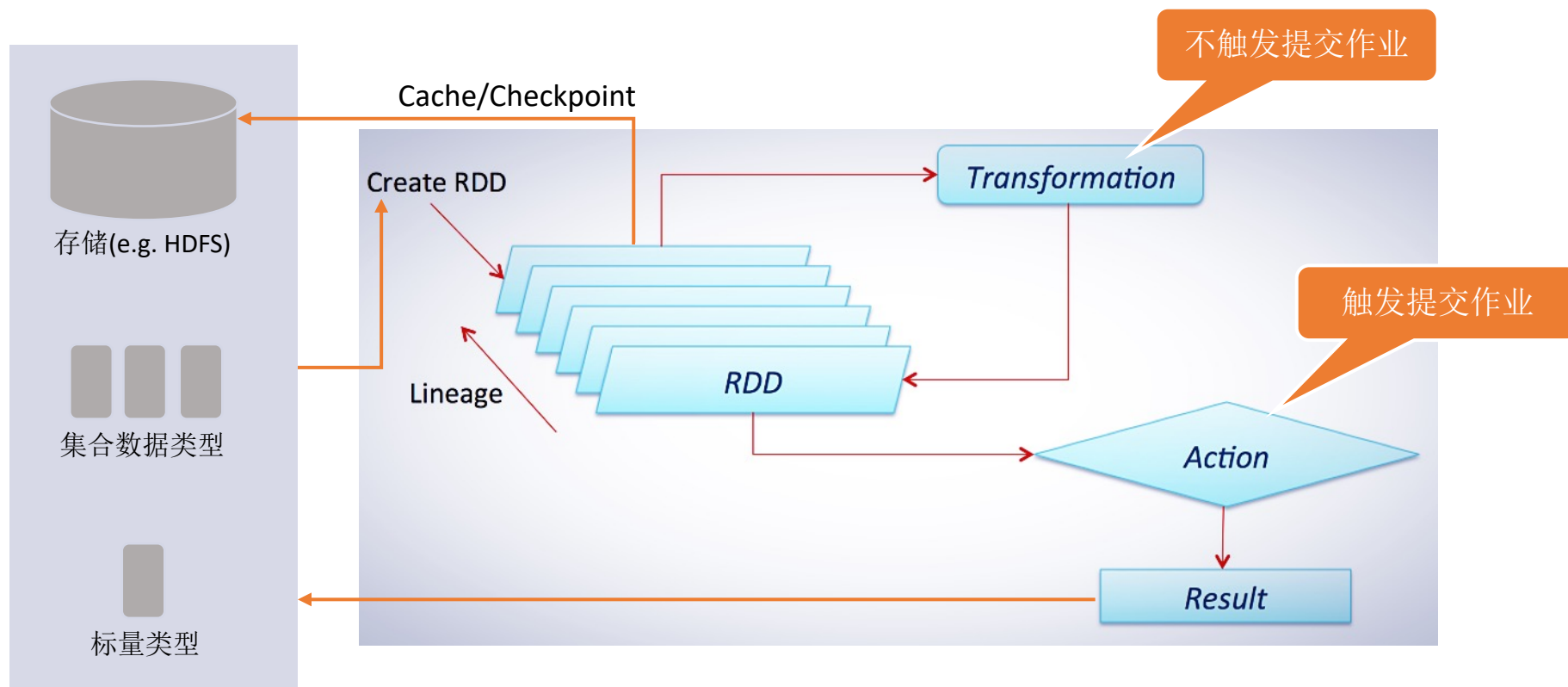
- 例如JdbcRDD

```
def map[U: ClassTag](f: T => U): RDD[U] = withScope {  
  val cleanF = sc.clean(f)  
  new MapPartitionsRDD[U, T](this, (context, pid, iter) => iter.map(cleanF))  
}
```



Spark——内核机制解析——RDD算子

- RDD算子运算流程



构造DAG，划分Job、Stage、Task，遇到Action，才会提交Job

Spark——内核机制解析——RDD算子

• Transformation

Transformation操作	说明
map(func)	对源RDD中的每个元素调用func，生产新的元素，这些元素构成新的RDD并返回
flatMap(func)	与map类似，但是func的返回是多个成员
filter(func)	对RDD成员进行过滤，成员调用func返回True的才保留，保留的构成新RDD返回
mapPartitions(func)	和map类似，但是func作用于整个分区，类型是Iterator<T> => Iterator<T>
mapPartitionsWithIndex(func)	...
union(otherDataset)	...
reduceByKey(func, [numTasks])	对<key, value>结构的RDD聚合，相同key的value调用reduce, func是(v,v)=>v
join(otherDataset, [numTasks])	...

• Action

Action操作	说明
reduce(func)	对RDD成员使用func进行reduce，func接收两个值返回一个值；reduce只有一个返回值。func会并发执行
collect()	将RDD读取到Driver程序里面，类型是Array，要求RDD不能太大
count()	返回RDD成员数量
first()	返回RDD第一个成员，等价于take(1)
take(n)	...
saveAsTextFile(path)	把RDD转换成文本内容并保存到指定的path路径下，可能有多个文件；path可以是本地文件系统路径，也可以是HDFS路径，转换方法是RDD成员的toString方法
saveAsSequenceFile(path)	...
foreach(func)	对RDD的每个成员执行func方法，没有返回值



Spark——内核机制解析——RDD依赖

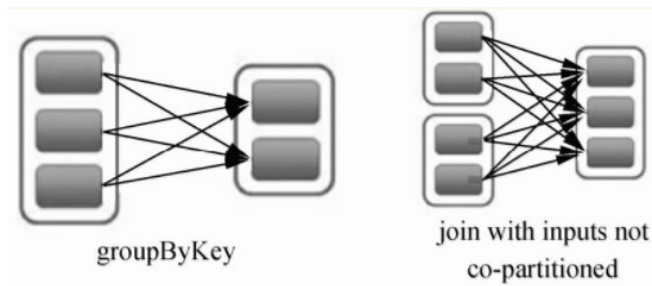
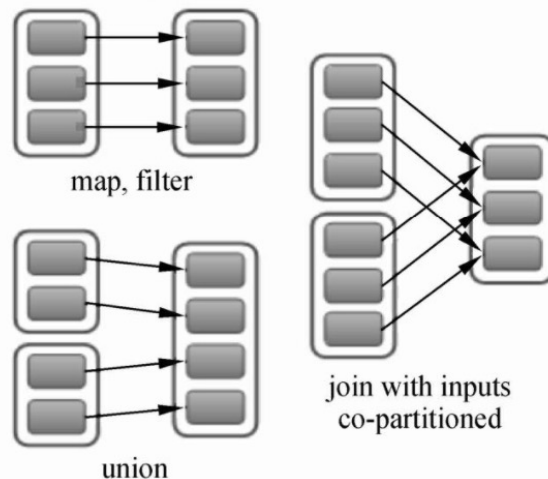
- 窄依赖NarrowDependency

- 每一个父RDD中的Partition最多被子RDD的1个Partition所使用
- 完全并行执行
- 哪些Transformation是窄依赖？

- 宽依赖ShuffleDependency

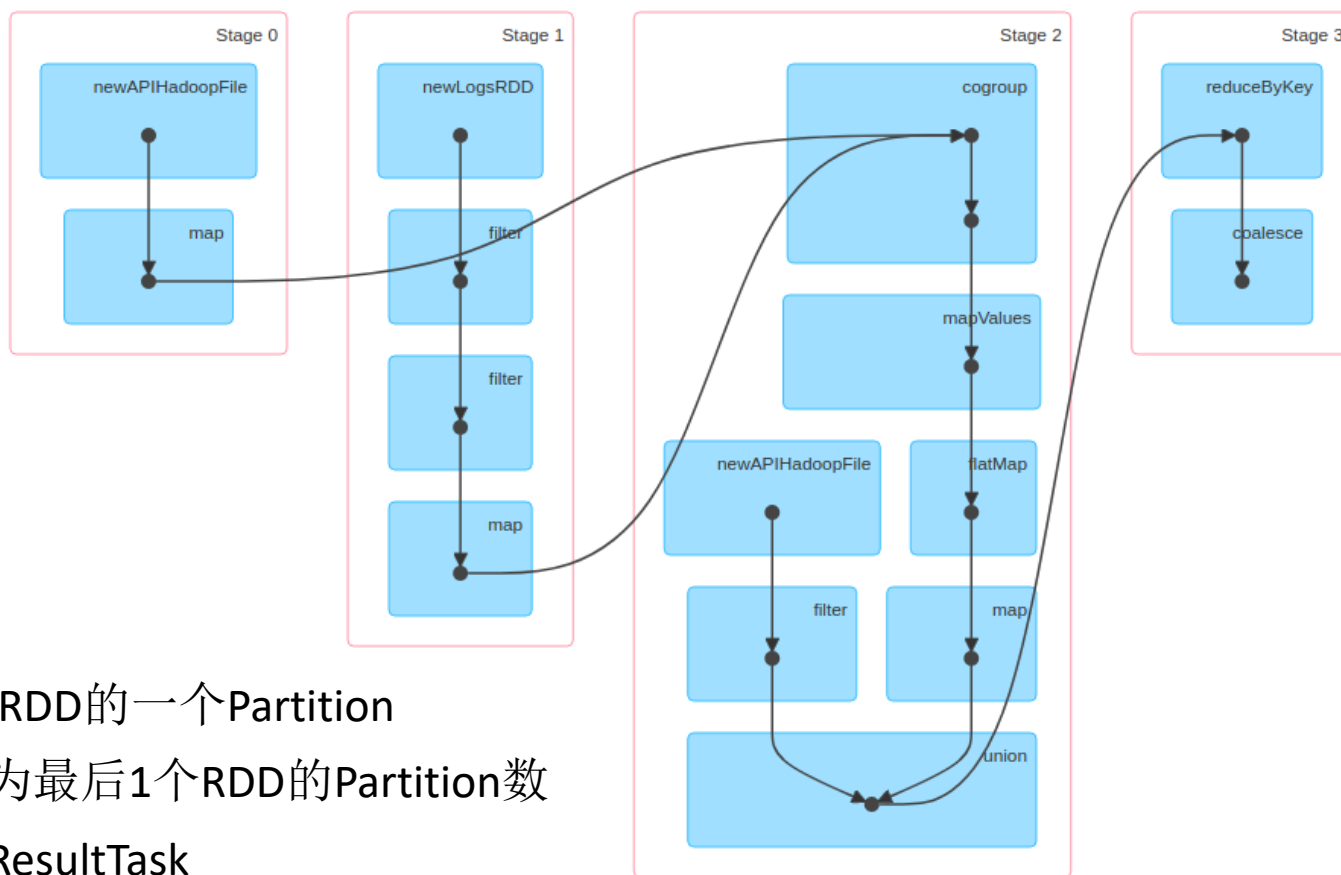
- 一个父RDD的Partition会同时被子RDD的多个Partition所使用
- 需要Shuffle后才能对计算并行化；Shuffle过程不能完全并行
- 哪些Transformation是宽依赖？

Narrow Dependencies:



Spark——内核机制解析——DAG

- DAG实例



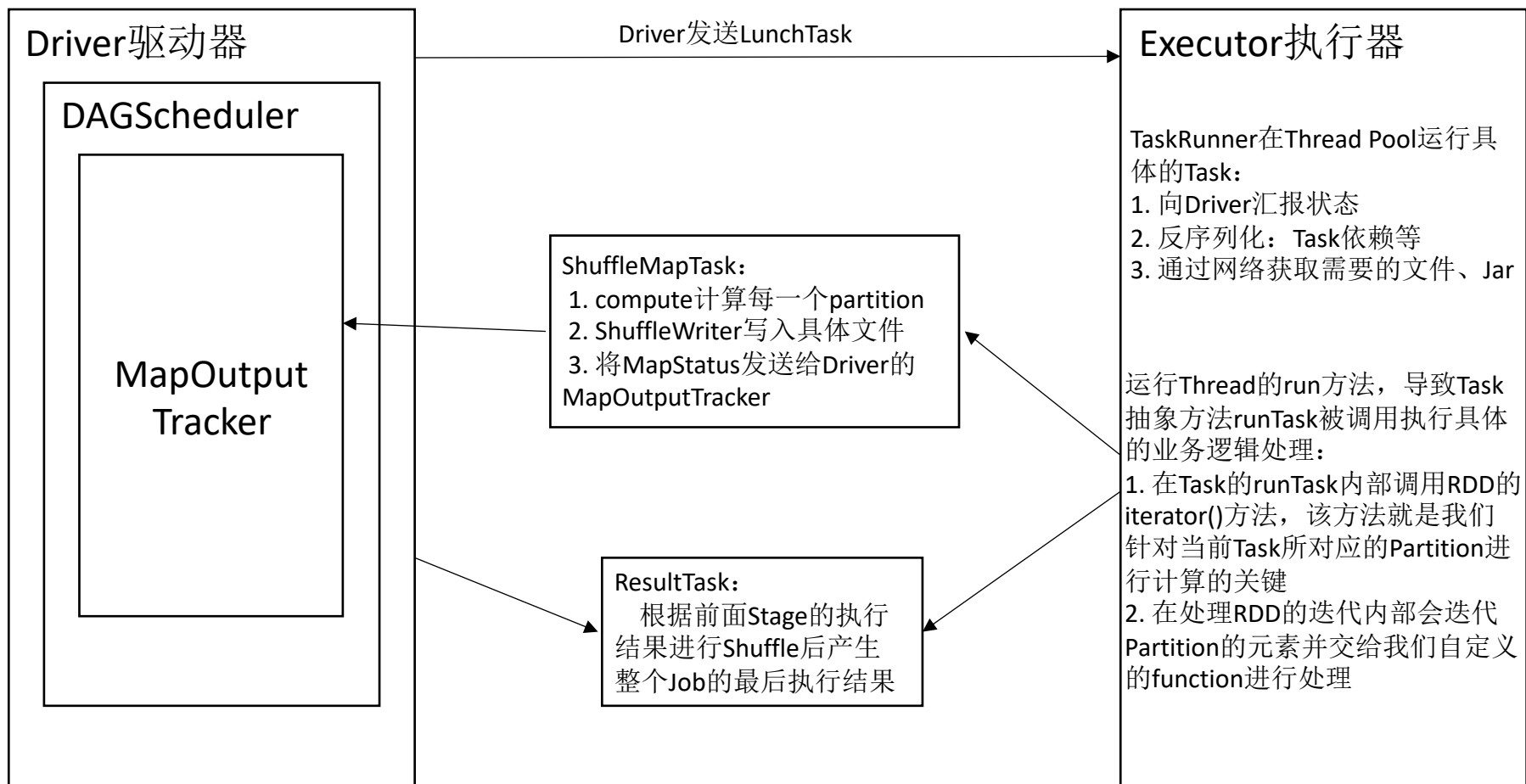
1个Task负责处理RDD的一个Partition

Stage的Task个数为最后1个RDD的Partition数

ShuffleMapTask, ResultTask



Spark——内核机制解析——RDD计算过程



Spark——内核机制解析——RDD缓存和检查点

- Cache：将RDD保存到内存（也可能在磁盘上），读写速度极快
 - 实现方式是在Persist持久化时将StorageLevel设置为MEMORY_ONLY
 - 共12种StorageLevel；本质上和Persist没有区别，都是Persist方法，只是级别不同
 - 作用：计算RDD完成后对其进行缓存，若整个计算失败，可以从缓存读取这个RDD，避免重新计算这个RDD，提升容错效率
 - 适用场景：
 - 获取大量数据后
 - 进行一个非常长的计算链条，设置一些缓存点
 - 某个步骤计算非常耗时，步骤完成后对结果进行缓存
 - 进行checkpoint前也会缓存
- Checkpoint
 - 将RDD持久化到HDFS，利用HDFS的容错能力降低RDD数据丢失的风险
 - 会将RDD的依赖清空，如果HDFS也不能保证数据不丢失，则任务需要重新启动



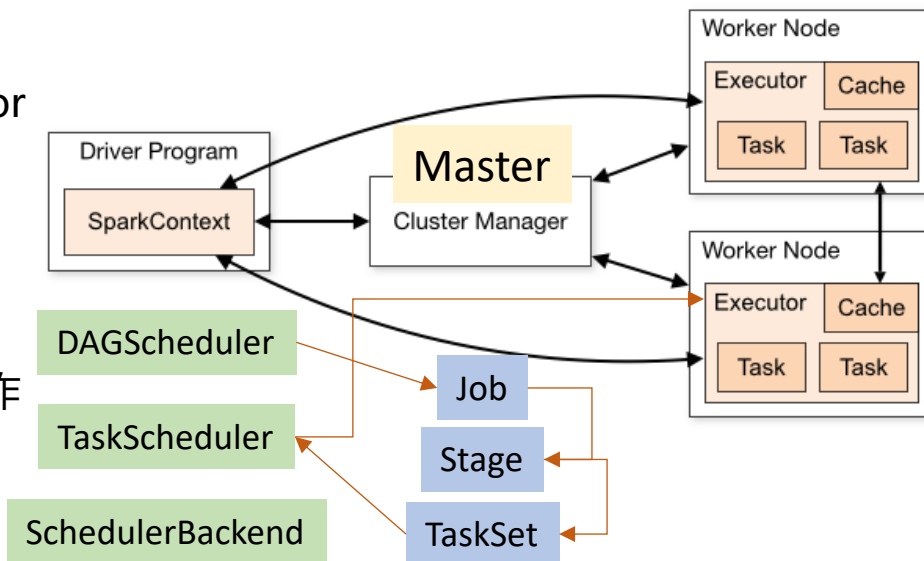
Spark——内核机制解析——RDD容错总结

- 4个不同的层次
 - Stage输出失败，上层调度器DAGScheduler重试
 - Task内部任务失败，底层TaskScheduler调度器重试
 - 根据RDD Lineage血统重新计算
 - 对于宽依赖而言，如果结果的一个Partition出错，需要重新计算父RDD所有Partition
 - 对于窄依赖而言，如果结果的一个Partition出错，只需重新计算父RDD被丢失Partition依赖的那个Partition
 - Cache可看作时Lineage容错机制的效率提升机制，本身并无容错考虑，如果Cache丢失则仍需重新计算
 - Checkpoint机制
 - 清空了所有依赖，依靠HDFS的容错能力



Spark——内核机制解析——Scheduler

- 调度器简洁清晰和高效
 - 输入：Spark RDD；输出：执行器Executor
- 核心对象RDD回顾
 - 是只读的内存数据，可以持久化
 - 是可以分区的数据集合
 - 是一个对象，可以调用transformation操作
 - 是可以通过重复执行变换操作恢复的
 - 变换操作时延迟操作
 - 主要两种创建方式：从持久化数据创建，从其他RDD变换操作创建
- 运行RDD计算的框架和组件
 - SparkContext
 - DAGScheduler, TaskScheduler, SchedulerBackend
 - Job, Stage, Task
 - Master, Worker
 - Executor, Driver

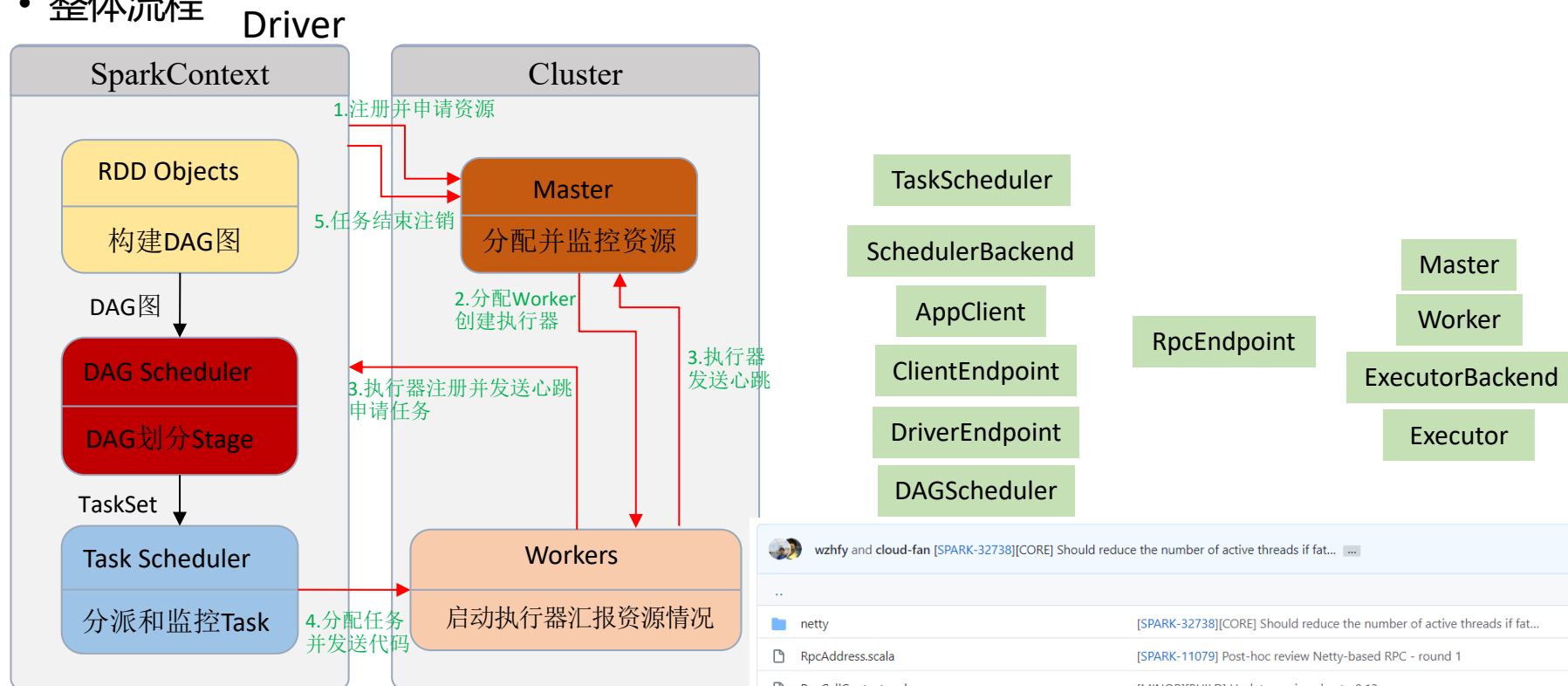


Spark Driver是运行Application的main函数
负责初始化SparkContext，它负责
与集群通讯、资源申请、任务分配和监控等。



Spark——内核机制解析——Scheduler

• 整体流程



wzhfy and cloud-fan [SPARK-32738][CORE] Should reduce the number of active threads if fat...	
netty	[SPARK-32738][CORE] Should reduce the number of active threads if fat...
RpcAddress.scala	[SPARK-11079] Post-hoc review Netty-based RPC - round 1
RpcCallContext.scala	[MINOR][BUILD] Update genjavadoc to 0.13
RpcEndpoint.scala	[SPARK-29398][CORE] Support dedicated thread pools for RPC endpoints
RpcEndpointAddress.scala	[SPARK-19365][CORE] Optimize RequestMessage serialization
RpcEndpointNotFoundException.scala	[SPARK-6028] [CORE] Remerge #6457: new RPC implementation and also pick ...
RpcEndpointRef.scala	[SPARK-31472][CORE] Make sure Barrier Task always return messages or ...
RpcEnv.scala	[SPARK-21408][CORE] Better default number of RPC dispatch threads.
RpcEnvStoppedException.scala	[SPARK-12967][NETTY] Avoid NettyRpc error message during sparkContext...
RpcTimeout.scala	[SPARK-27404][CORE][SQL][STREAMING][YARN] Fix build warnings for 3.0...



Spark——内核机制解析——Scheduler

- DAGScheduler构建Stage
 - 在createStage时从最后一个Stage开始递归构建Stage，并根据dependency类型划分Stage

CreateResultStage

获取当前Stage边界

根据每一个边界构造父Stage

获取当前Stage边界

根据每一个边界构造父Stage

获取当前Stage边界

根据每一个边界构造父Stage

新建Stage并作为父Stage返回

新建Stage并作为父Stage返回

新建Stage并作为父Stage返回

ShuffleMapStage

新建ResultStage



Spark——内核机制解析——Executor

- 创建Executor

- 首先Application要向Master注册—— 由SchedulerBackend通过AppClient的子类ClientEndpoint完成
- Master回复注册成功，并将Application发送到满足条件的Worker上，在Work上启动Executor
- 通过worker的RpcEndpoint向Worker发送启动Executor请求
- Worker收到请求后启动ExecutorRunner，并通知Master Executor的相关信息 Executor没有真的启动
- 此时本质上是启动了负责帮Executor和Driver通信的Backend：ExecutorBackend
- ExecutorBackend直接向Driver注册Worker让它准备启动的Executor
- 收到Driver确认后，Executor正式新建

是Executor主动联系Driver，让SchedulerBackend了解到所有Executor的信息，再让TaskScheduler根据Executor情况分配Task到Executor

- Executor资源分配

- Driver启动的时候从spark-submit收集对executor的需求，并在向Master注册时发送过去
- 具体可以参考如何从提交程序收集需求，并一直传递给Master，再给Worker
- ExecutorRunner从Worker那里得到这些参数，并在根据参数启动ExecutorBackend JVM进程

- Executor启动

- 在ExecutorBackend向Driver注册后，收到确认时的操作就是创建Executor



Spark——内核机制解析——Executor

- 此时
 - TaskScheduler根据Executor情况安排好TaskSet调度
 - Executor也启动好了
- 通过DriverEndpoint向ExecutorBackend发送执行Task的消息

```
executorData.executorEndpoint.send(LaunchTask(new SerializableBuffer(serializedTask)))
```

- ExecutorBackend收到LaunchTask通知后
 - 调用Executor的launchTask方法
 - 内部创建TaskRunner，用线程池执行taskrunner，调用run方法
 - 反序列化出Task
 - 调用Task的run方法

<https://github.com/apache/spark/tree/master/core/src/main/scala/org/apache/spark/>

master spark / core / src / main / scala / org / apache / spark /			Go to file
Ngone51 and cloud-fan [SPARK-32850][CORE] Simplify the RPC message flow of decommission			56ae958 16 hours ago History
			2020.09.17
api	[SPARK-32398][TESTS][CORE][STREAMING][SQL][ML] Update to scalatest 3...	2 months ago	
broadcast	[SPARK-32715][CORE] Fix memory leak when failed to store pieces of br...	2 days ago	
deploy	[SPARK-32850][CORE] Simplify the RPC message flow of decommission	16 hours ago	
executor	[SPARK-32850][CORE] Simplify the RPC message flow of decommission	16 hours ago	
input	[SPARK-29291][CORE][SQL][STREAMING][MLLIB] Change procedure-like decl...	12 months ago	
internal	[SPARK-32736][CORE] Avoid caching the removed decommissioned executor...	9 days ago	
io	[SPARK-29322][CORE] Enable closeFrameOnFlush on ZstdOutputStream for ...	12 months ago	
launcher	[SPARK-27323][CORE][SQL][STREAMING] Use Single-Abstract-Method suppor...	2 years ago	



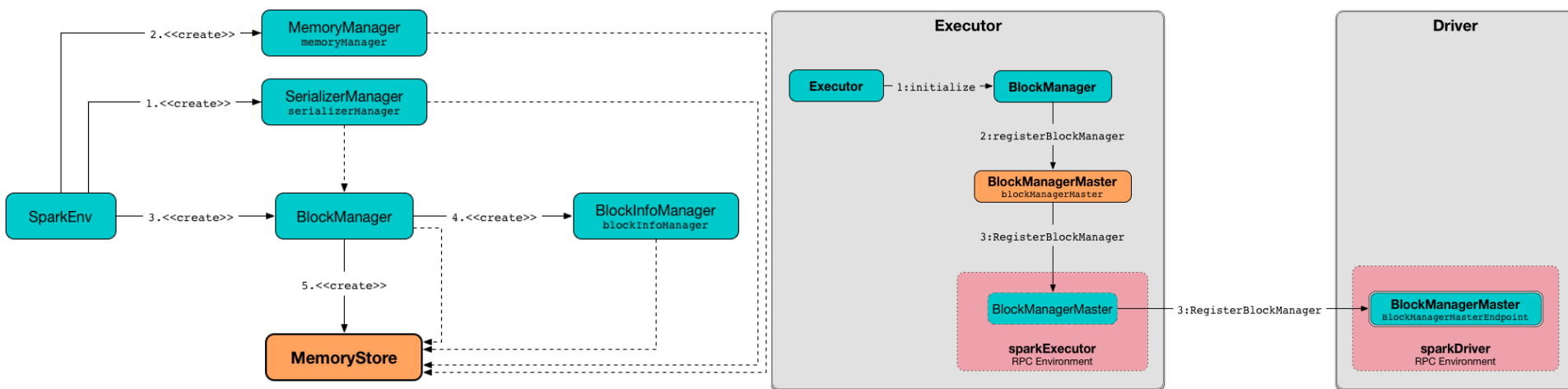
Spark——内核机制解析——Storage

- 通信层

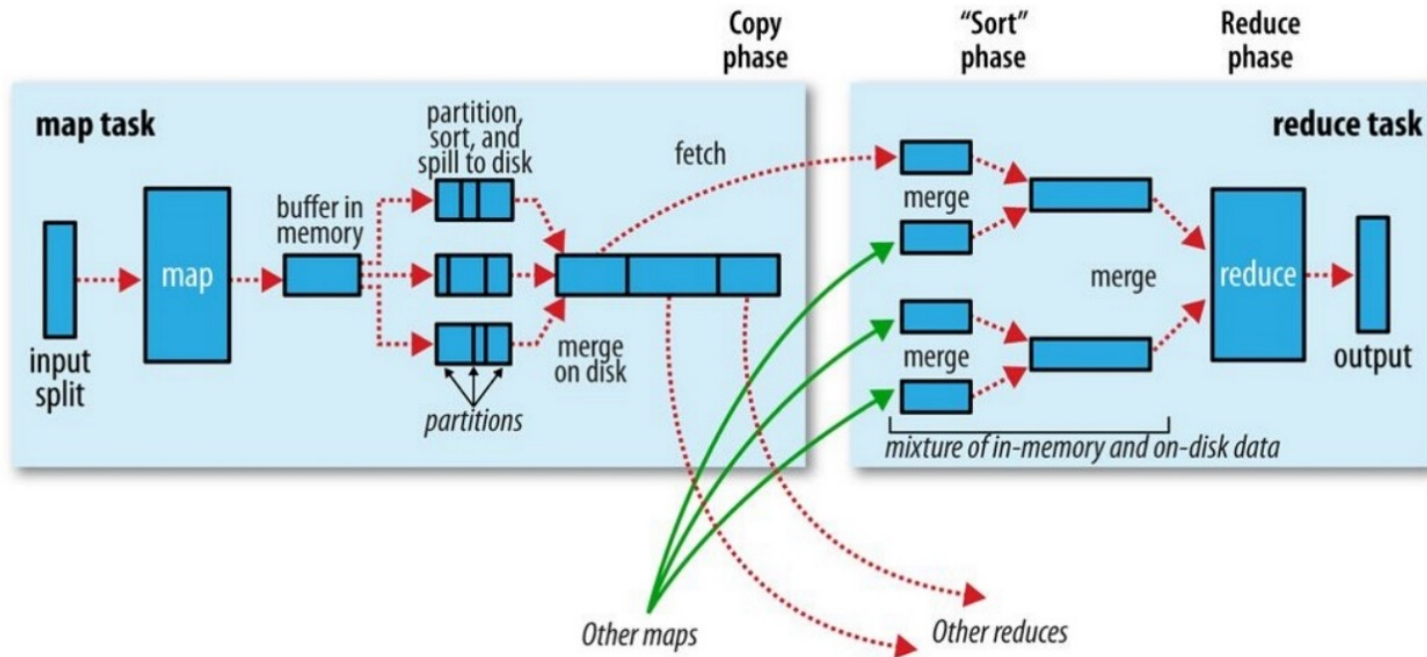
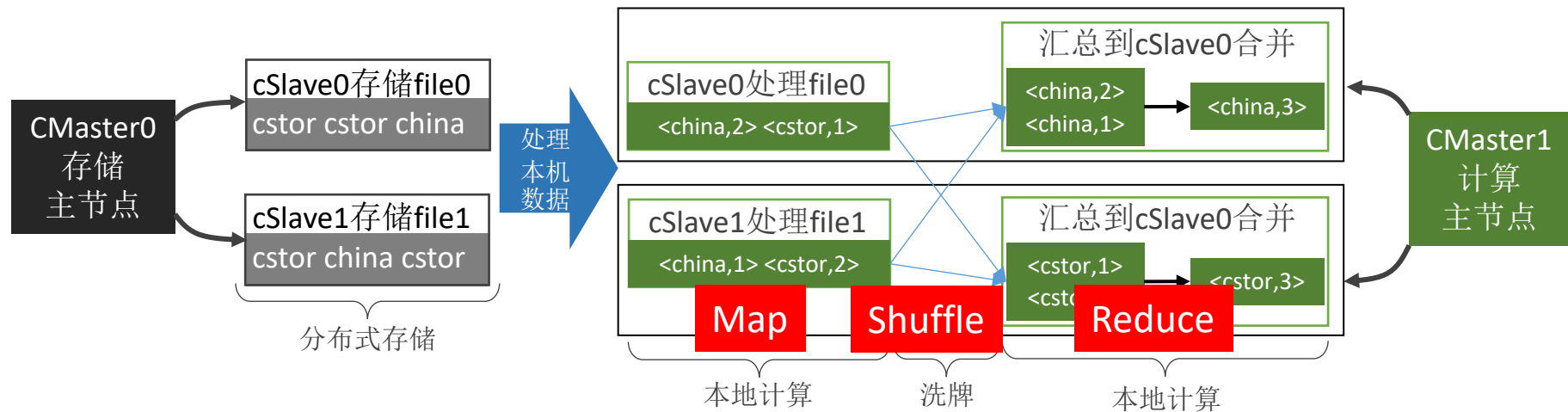
- Master-Slave结构, 传输控制和状态信息
- BlockManager**, BlockManagerMaster, BlockManagerMasterEndpoint, BlockManagerSlaveEndpoint

- 存储层

- 负责将数据存储到内存、磁盘或堆外内存中, 为数据在远程节点生成副本
- DiskStore, MemoryStore



Spark——内核机制解析——Shuffle



Spark——内核机制解析——Shuffle框架

- 可插拔的Shuffle框架
 - ShuffleDependency, ShuffleManager, ShuffleHandle
 - ShuffleReader, ShuffleWriter
 - ShuffleMapTask, ResultTask
- 【1】构建RDD时确定好与父RDD之间的依赖关系，如果是ShuffleDependency，会向ShuffleManager注册，获取Handle，用来保存父RDD的相关信息 **看RDD创建过程**
- 【2】在实际计算时，runTask会调用RDD compute方法，其中根据dependency获取reader读取用来计算RDD的输入数据，也就是之前shuffle操作写入的数据 **看RDD计算过程**
- 【3】当runTask完成计算后，获取子RDD之间的依赖关系，如果是shuffle依赖则同样通过dependency和ShuffleManager获得Handle，进而获得writer **看Task结束的处理**



Spark——内核机制解析——Shuffle机制演进

- Shuffle前后必不可少地需要网络I/O，通过数据序列化方法和压缩技术进行效率优化
 - Spark中序列化方法和压缩算法的配置
- Spark 1.0：基于Hash的Shuffle机制
 - 每一个Mapper阶段的Task都会为Reduce阶段的每一个Task生成1个文件， $M \times R$ 个
 - 合并机制：每一个执行单位为Reduce阶段每一个Task生成1个文件
- Spark 1.1：基于Sort的Shuffle机制
 - 每一个Mapper阶段的Task生成两个文件：索引和数据文件，Reduce阶段通过索引读取
- Spark 1.4：钨丝计划——优化内存管理模型
 - 直接使用二进制数据，而不是Java对象；避免序列化和反序列化开销



目录

- 分布式计算
 - 分布式计算概述
 - 分布式计算理论基础
- 分布式系统
 - 特性
 - 类型
- 分布式计算、存储和资源管理系统
 - 案例1: Hadoop
 - 案例2: Spark
 - 案例3: 阿里开放数据处理平台



阿里巴巴开放数据处理服务

- 开放数据处理服务ODPS (Open Data Processing Service)
 - 基于飞天平台实现，用于海量数据存储和计算的服务
 - 应用
 - 淘宝、天猫双十一：每秒创建订单8万，每小时处理数据17PB，当天交易总额571亿（2014年）
 - 余额宝：客户数量接近1.5亿，规模突破5000亿元，每秒处理11000笔请求
 - 蚂蚁金服：1秒审批，小微贷款坏账率<1%（截至2015年1月），累计贷款30万家



需求和设想

- 2009年的状态

存

- IOE + Greenplum + Hadoop +
 - 存储昂贵，可扩展性差

通

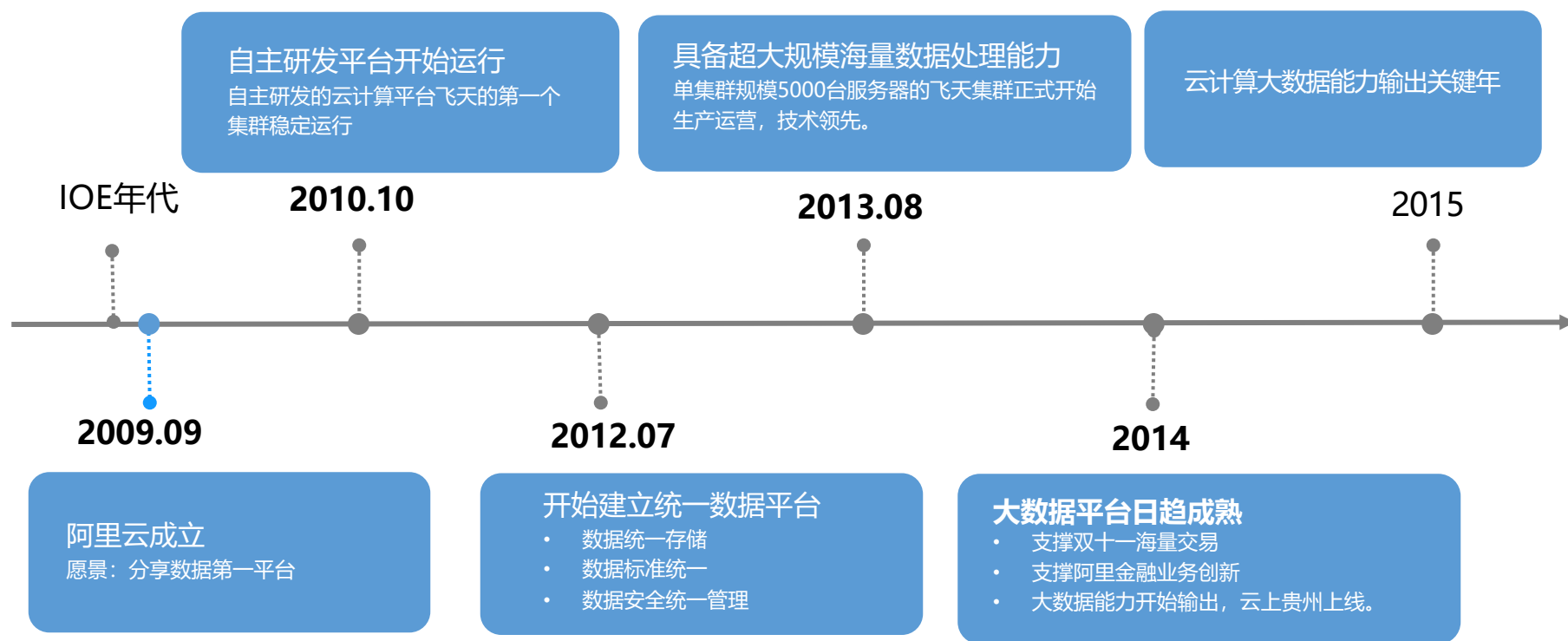
- 数据孤岛
 - 各业务部门的数据散落在多个集群，彼此之间数据不通，数据共享太难

用

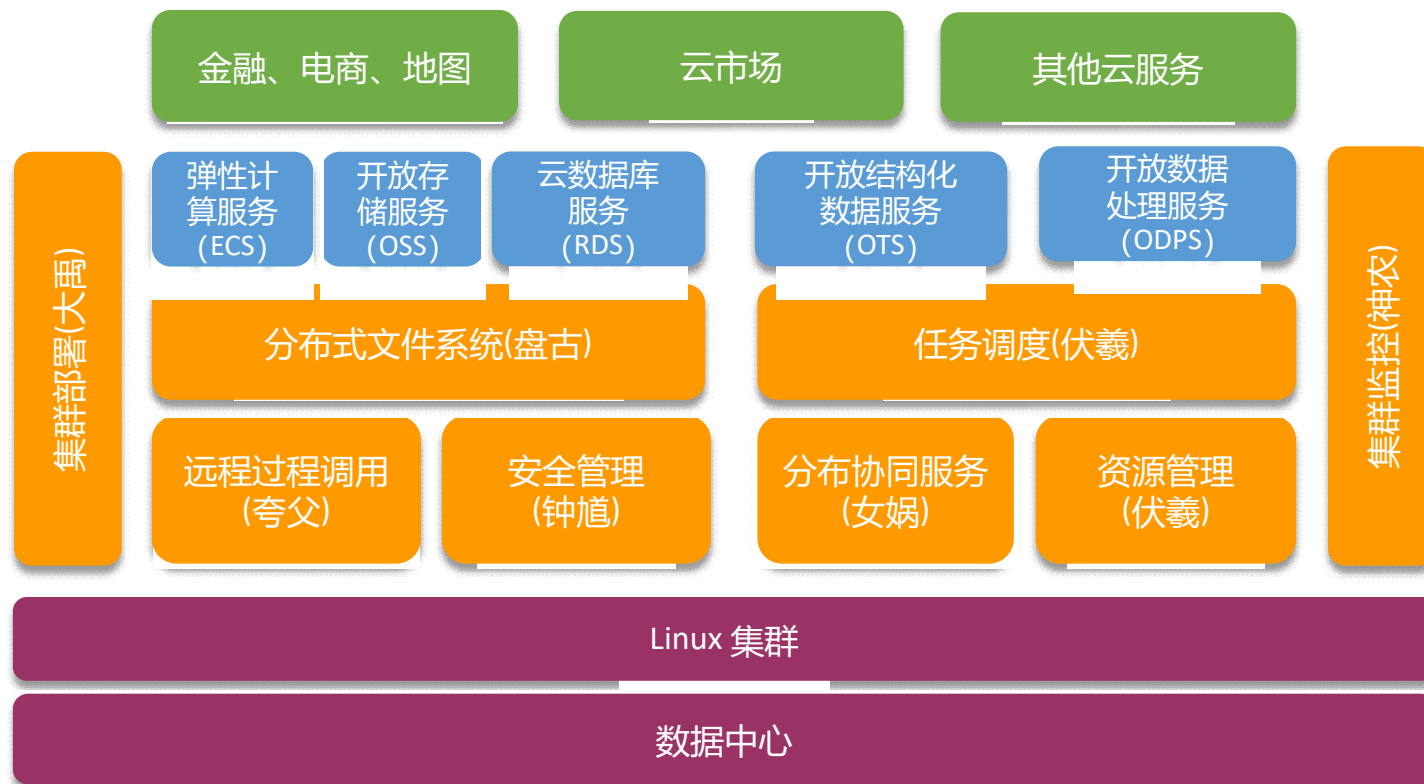
- 数据重复建设
 - 由于数据不集中，导致数据被重复存储和计算，光淘宝商品类目表就有70多张.....



阿里大数据平台发展历程

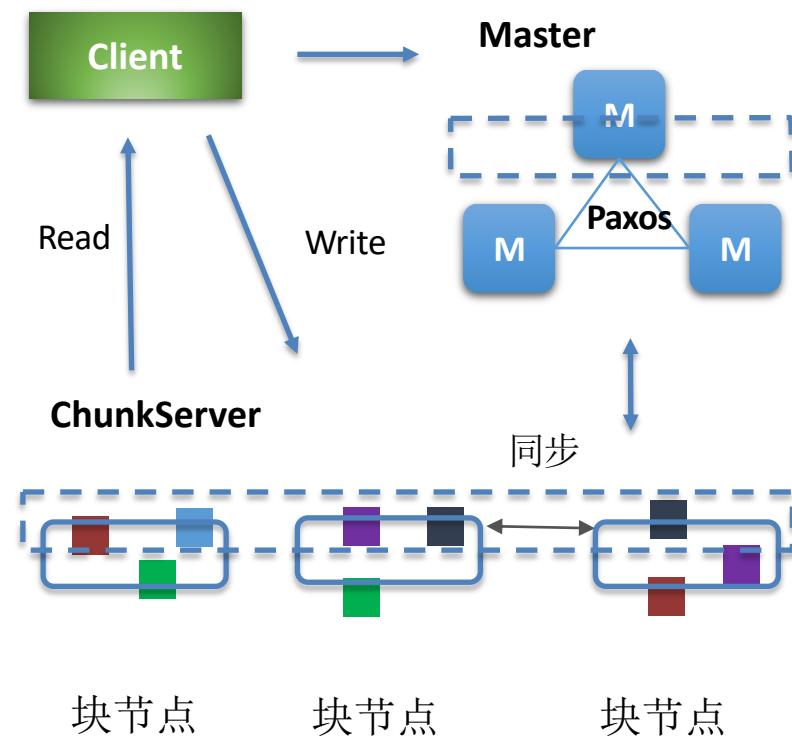


飞天开放平台



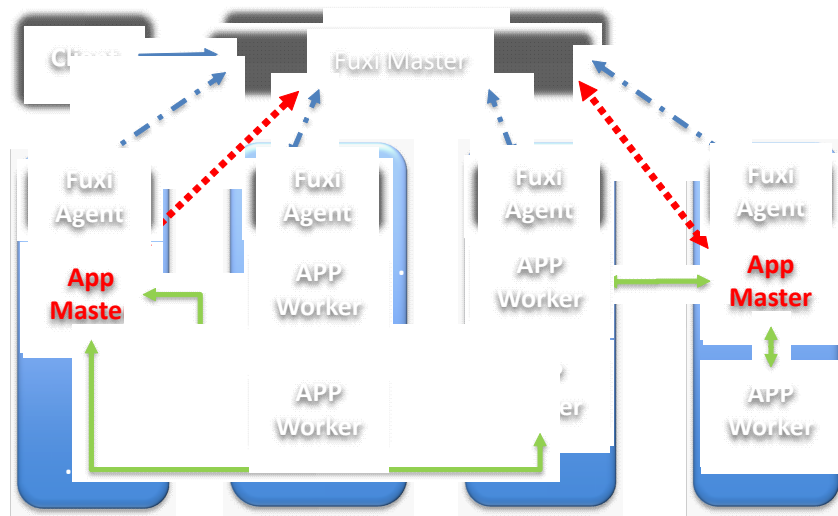
分布式文件系统（盘古）

- Master/Slave架构
- 稳定性增强
 - 基于Paxos的多Master架构，自动故障恢复时间小于1分钟，透明热升级
- 多租户增强
 - Capability与目录配额，流控、优先级与公平性，离线/在线混布
- 性能增强
 - 混合存储，原生RaidFile支持
 - 锁优化，读写分离
- 规模增强
 - 文件数无限制，单集群 > 5K

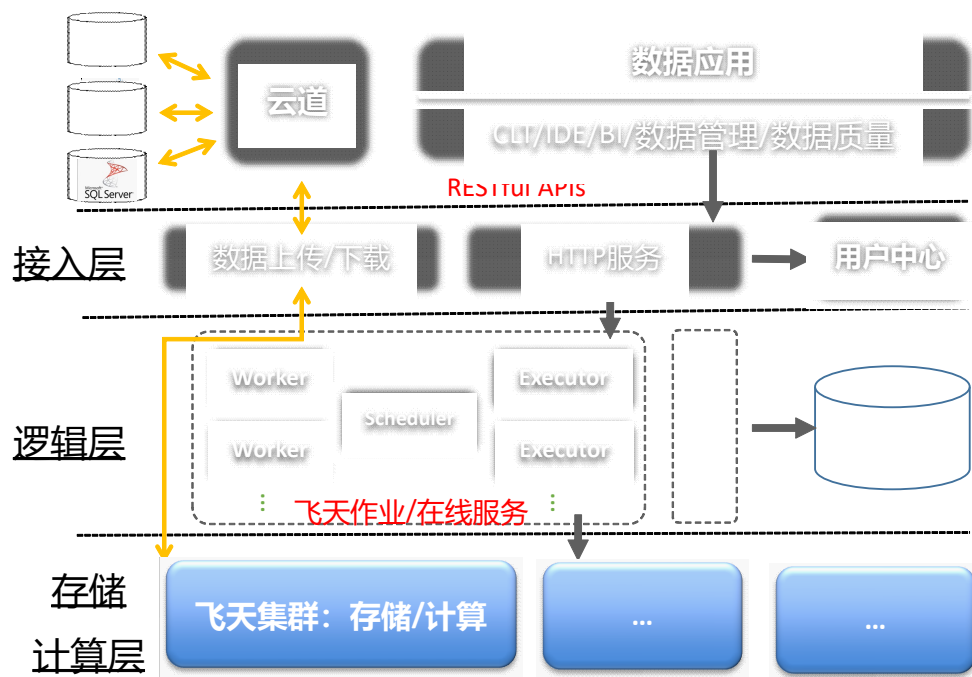


资源管理与任务调度（伏羲）

- 稳定性增强
 - Fuxi Master HA, AppMaster Failover
- 多租户增强
 - 多维度资源：CPU/内存..., 配额组管理, 弹性（min/max）
 - Cgroup隔离, 进程沙箱
 - 离线/在线混布
- 规模增强
 - 增量调度, 10K+规模
- 编程模型：Job/Service



ODPS



- 单一集群规模可以达到10000+服务器（保持80%线性扩展）
- 单个ODPS部署可以支持100万服务器以上，无限制（线性扩展略差），支持同城、异地多数据中心模式
- 10000+用户数，1000+项目应用、100+部门（多租户）
- 100万以上作业（目前单日平均提交任务），20000以上并发作业

ODPS的丰富功能

- 海量数据存储
 - 多份拷贝，突破单一集群限制，增加存储利用率
- 丰富的计算工具和编程模型
 - SQL：语法兼容Hive，函数语义与传统关系数据库更兼容
 - 流计算
 - MapReduce
 - 图计算：PageRank，K-均值聚类，金融风控，
 - 算法平台：SVD分解，逻辑回归，随机森林，朴素贝叶斯，



总结

- 什么是分布式计算——广义上的计算
- 分布式计算理论基础——ACID, CAP, BASE, 一致性算法, 一致性 hash
- 分布式系统——特性、类型
- 案例【Hadoop, Spark, 阿里】



谢 谢



南京大學
NANJING UNIVERSITY